

SuloFut

Készítette: Kakas Zalán, Bugja Botond

13.D

2026.05.04

Tartalomjegyzék

1.	Bevezetés.....	4
1.1	A téma ismertetése	4
1.2	Témaválasztás indoklása	4
1.3	Szakmai célok	4
2.	Fejlesztői dokumentáció.....	5
2.1	Adatszerkezetek és adattárolás	5
2.1.1	Relációs adatbázis (SQL).....	5
2.1.2	JSON alapú adattárolás.....	7
2.2	A program	10
2.2.1	A program folyamatábrája.....	11
2.2.2	A program felépítése.....	11
2.3	A program különlegesebb algoritmusai	14
2.3.1	Különlegesebb algoritmusok a backend-en	14
2.3.2	Különlegesebb algoritmusok a frontend-en	36
2.4	A program tesztelése.....	48
2.4.1	Backend teszt.....	48
2.4.2	Frontend teszt.....	51
2.5	Továbbfejlesztési lehetőségek.....	53
3.	Felhasználói dokumentáció	55
3.1	A program célja	55
3.2	Rendszer követelmények.....	55
3.2.1	Processzor (CPU).....	55
3.2.2	Videókártya (GPU).....	55
3.2.3	Memória (RAM)	56
3.2.4	Hálózat és Böngésző.....	56
3.3	Telepítés.....	56
		59

3.4	Használat	61
3.5	Fő funkciók.....	61
3.5.1	Saját felhasználó.....	61
3.5.2	Squad	63
		63
		63
3.5.3	Squad Building Challenge.....	64
		64
		64
3.5.4	Draft.....	65
3.5.5	Store	67
3.5.6	Club	69
3.5.7	Filter.....	69
3.5.8	Leaderboard	70
3.5.9	Objectives.....	70
3.5.10	Settings	72
4.	Csoportmunka megvalósítása.....	73
4.1	A kód.....	73
4.2	Egyebek.....	73
4.3	Menete	73
5.	Összegzés	74
5.1	A kitűzött célok teljesítése	74
5.2	Nehézségek a megvalósításban.....	74
6.	Ábrajegyzék.....	76
7.	Irodalomjegyzék.....	79

1. Bevezetés

1.1 A téma ismertetése

- A választott témánk egy, a valós futballon alapuló mégis kicsit fantáziájában elrugaszkodottabb játék. Különböző játékmódok és funkciók szórakoztathatják a játékosokat, legyen foci szurkoló vagy olyan, aki életében nem nézett focit.

- Évek hiánya egy teljes foci kártyákon alapuló játékra, ami tartalmazza az összes funkciót felépült bennünk és ide jutottunk, hogy lehetőségünk van megépíteni ezt és ezért éltünk is a lehetőséggel.

1.2 Témaválasztás indoklása

- Kiskorunk óta jászunk ilyen és ehhez hasonló játékokkal, a foci nagyjából a szívünkben él, amikor először eszünkbe jutott az ötlet félig viccként gondoltuk, aztán rájöttünk, hogy képesek lennénk erre. Úgy gondoltuk/gondoljuk teljesen testhez illő volt ez a téma számunkra.

1.3 Szakmai célok

- Célunk volt egy új frontend nyelv megtanulása a React.js formájában, ezt a célunkat sikerült elérni, a jelenlegi ismert programozási nyelvekből fejlődés mellett. Illetve grafikus képességeket is sikerült fejleszteni, ez megjelenik a kártyák kinézetében, és sok más különböző grafikai elemekben is. A használt programozási nyelvek (HTML 5, BOOTSTRAP 5, CSS, Node.js, React.js, JS) összhangjával alkottuk meg az oldalt.

2. Fejlesztői dokumentáció

2.1 Adatszerkezetek és adattárolás

- A projekt fejlesztése során többféle adattárolást alkalmaztunk annak érdekében, hogy minél gyorsabban és hatékonyabban tudjon működni a játékunk. Az adatok két fő csoportra bonthatók: relációs adatbázisban (SQL) tároljuk a “dinamikus” (és egy-két statikus) adatokat, és JSON formátumban tároljuk a statikus, vagy nagyon ritkán változó adatokat.

2.1.1 Relációs adatbázis (SQL)

- A felhasználókhöz és a játékmenethez kapcsolódó adatokat ilyen módon tároljuk. Az adatbázis-kezeléshez SQL alapú rendszert használunk, amely lehetővé teszi az adatok struktúrált kezelését.

2.1.1.1 Az adatbázis táblái

- Adminhoz tartozó adatok

- admin tábla -> admin bejelentkezéshez szükséges adatok

- Felhasználókhöz tartozó adatok

- users tábla -> felhasználói bejelentkezéshez szükséges adatok
- stats tábla -> felhasználók elért sikerei (ranglistához)
- userclub tábla -> felhasználók klub-ja
- userpacks tábla -> felhasználók paklijai
- user_sbc tábla -> felhasználók SBC-jeinek kitöltései
- user_subobjective_progress tábla -> felhasználók kihívás-előrehaladásai
- user_objective_claims tábla -> felhasználók kihívásaiért járó jutalmak begyűjtései

- Kihívásokhoz tartozó adatok (Objectives)

- objcategories tábla -> Kihívások kategóriái
- objectives tábla -> Kihívások csoportjai
- subobjectives tábla -> Alkihívások

- Csapatépítő kihívásokhoz tartozó adatok (SBC)

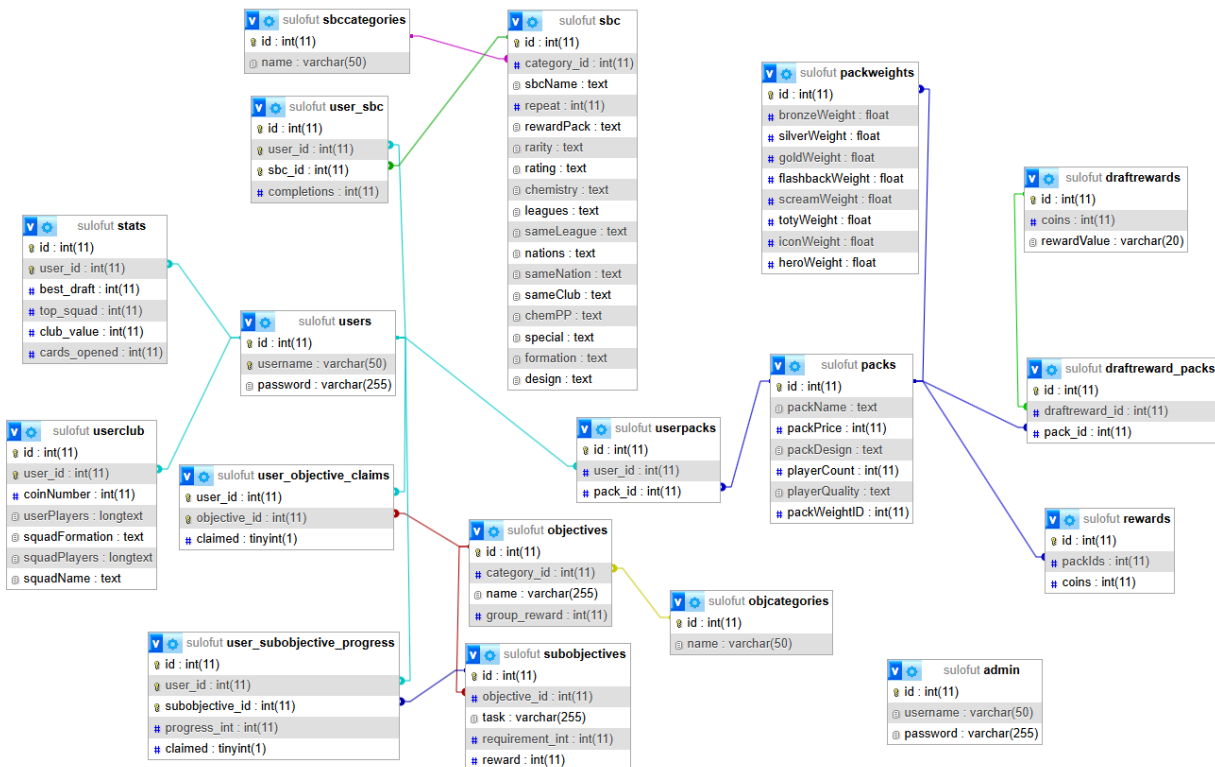
- sbccategories tábla -> SBC-k kategóriái
- sbc tábla -> SBC-k

- Paklikhoz és jutalmakhoz tartozó adatok

- draftrewards tábla -> Draft jutalmak és értékeik
- draftreward_packs tábla -> Draft jutalmak paklijai
- packs tábla -> Paklik
- packweights tábla -> Paklikban szereplő játékos rarity-k súlyozása
- rewards tábla -> Jutalmak

2.1.1.2 Adatbázis kapcsolatai

- Adatbázisunkban számos tábla került összekötésre a kapcsolatok biztosításához. Ezt az alábbi ábrán lehet megtekinteni:



Kép 1 - Sql relációk

2.1.2 JSON alapú adattárolás

- A játékhoz tartozó, ritkán, vagy egyáltalán nem változó úgy nevezett statikus adatokat JSON fájlokban tároljuk. Ez csökkenti az alap adatbázisra helyezendő terhelést, és egyszerűbbé teszi a későbbi új adatok beillesztését.

2.1.2.1 JSON fájlok

- players.json -> A játékban szereplő összes játékos adatait tartalmazza. Egy játékos objektum például a következőképpen néz ki:

```
{
  "player_id": "252371",
  "short_name": "J. Bellingham",
  "long_name": "Jude Victor William Bellingham",
  "player_positions": "CAM, CM",
  "overall": "90",
  "league_id": "53",
  "league_name": "La Liga",
  "league_level": "1",
  "club_team_id": "243",
  "club_name": "Real Madrid",
  "nationality_id": "14",
  "nationality_name": "England",
  "pace": "80",
  "shooting": "86",
  "passing": "83",
  "dribbling": "90",
  "defending": "78",
  "physic": "85",
  "player_face_url": "https://cdn.fifacm.com/content/media/imgs/fc26/players/p252371.png?v=26",
  "league_url": "https://www.fifacm.com/content/media/imgs/fc26/leagues/l53.png",
  "nation_url": "https://cdn.fifacm.com/content/media/imgs/fc26/nations/14.png?v=11",
  "club_team_url": "https://cdn.fifacm.com/content/media/imgs/fc26/teams/52/l243.png?v=11",
  "rarity": "gold",
  "value": 686
},
```

Kép 2 - Játékos JSON objektum

- formations.json -> A játékban szereplő összes formációt, és pozícióit, valamint azoknak helyét tartalmazza. Egy formáció objektum például a következőképpen néz ki:

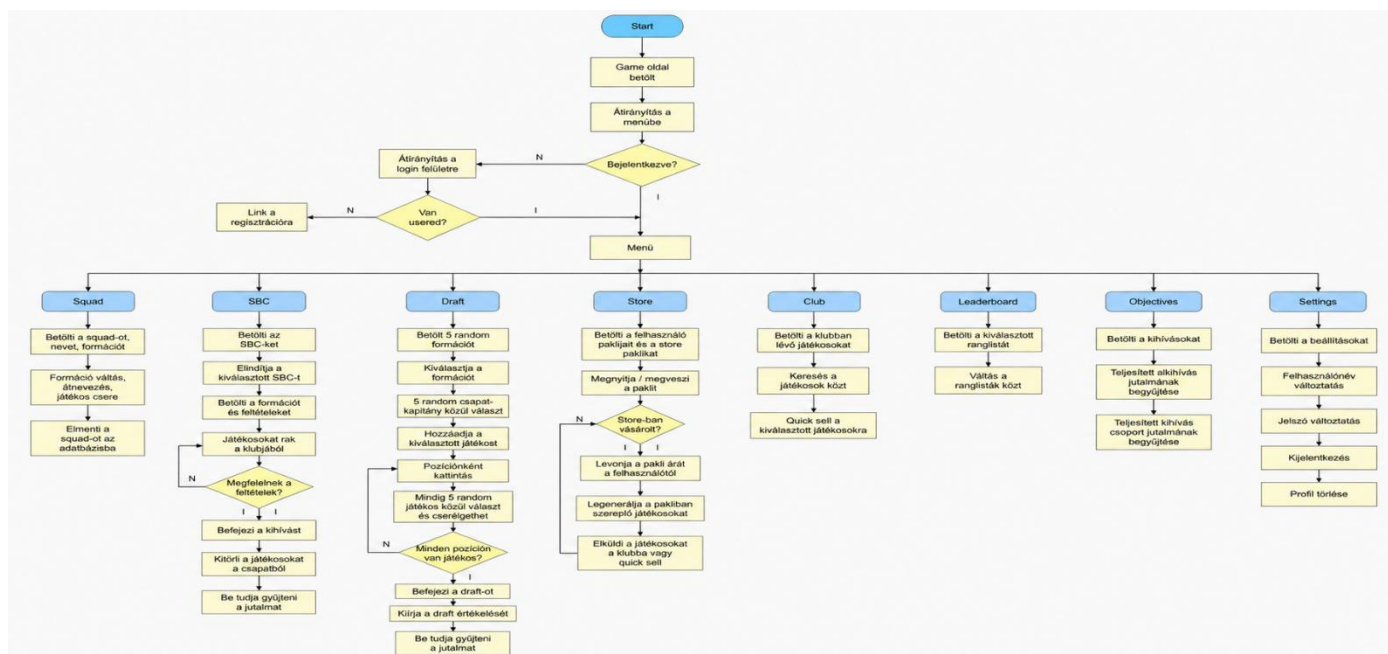
```
{
  "Formation": "3-1-4-2",
  "layout": [
    {
      "pos": "GK",
      "x": 50,
      "y": 87
    },
    {
      "pos": "CB",
      "x": 25,
      "y": 70
    },
    {
      "pos": "CB",
      "x": 50,
      "y": 70
    },
    {
      "pos": "CB",
      "x": 75,
      "y": 70
    },
    {
      "pos": "CDM",
      "x": 50,
      "y": 53
    },
    {
      "pos": "LM",
      "x": 20,
      "y": 43
    },
    {
      "pos": "CM",
      "x": 40,
      "y": 43
    },
    {
      "pos": "CM",
      "x": 60,
      "y": 43
    },
    {
      "pos": "RM",
      "x": 80,
      "y": 43
    },
    {
      "pos": "ST",
      "x": 35,
      "y": 17
    },
    {
      "pos": "ST",
      "x": 65,
      "y": 17
    }
  ]
},
```

Kép 3 – Formáció JSON objektum

2.2 A program

- A program fejlesztése során próbáltuk a komponensalapú szoftverfejlesztés elveit követni, mivel szerintünk nagyon hasznos, hogy egy jól megírt függvényt, vagy akár UI elemet bárhol fel tudjunk használni, ahol szükségünk van rá. Ez fontos hibakezelés szempontjából is, valamint a játék egységességének szempontjából is, arról nem is beszélve, hogy a forráskódban, valamint a tesztelés során is helyenként több tíz, akár több száz sort is megspóroltunk vele.
- Ehhez tökéletes volt számunkra a React könyvtára, és az általa kínált lehetőségek. A fejlesztési folyamat megkönnyítésére a Vite eszközt használtuk, amely gyors fejlesztői szerveret biztosított.

2.2.1 A program folyamatábrája



Kép 4 – A program folyamatábrája

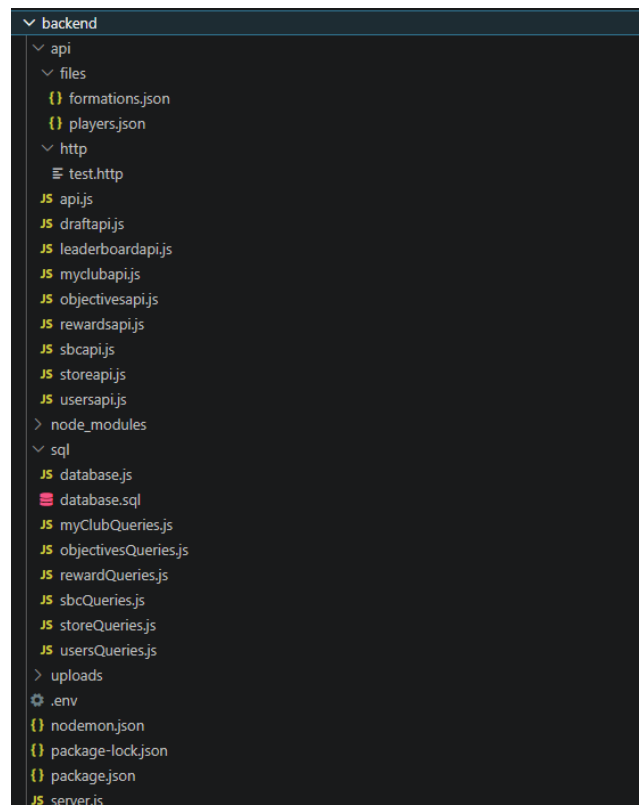
2.2.2 A program felépítése

- A program két fő részből áll, a backend és a frontend mappából, valamint ezekkel egy szintben van a .gitignore fájlunk, hogy mind a frontend-re, mind a backend-re vonatkozzon, valamint a README.md fájlunk, ami a játék elindításának lépéseit, valamint a játék elmagyarázását tartalmazza.

2.2.2.1 Backend mappa

- A backend mappában az alábbi fájlok találhatóak:

- api mappa -> a JSON fájlok, a test.http, és a végpontokat tartalmazó fájlok szerepelnek benne
- sql mappa -> a lekérdezéseket tartalmazó fájlok, valamint az adatbázis SQL fájlja szerepelnek benne
- uploads mappa -> az admin által feltöltött kártyadizájnok ide mentődnek
- .env fájl -> a session secret-et tartalmazza
- nodemon.json -> nodemon konfigurációt tartalmazza
- package-lock.json -> a projekt függőségeinek verzióit rögzíti
- package.json -> a projekt alap adatait, függőségeit, és futtatási beállításait tartalmazza
- server.js -> a projekt szerverének beállításait tartalmazza, valamint ez indítja el a szerveret



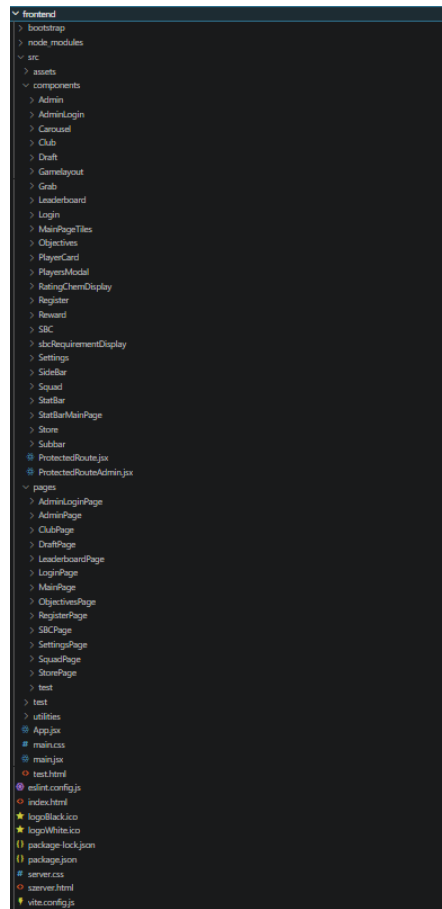
Kép 5 – A backend mappa

2.2.2.2 Frontend mappa

- A frontend mappában az alábbi fájlok találhatóak:

- bootstrap mappa -> a bootstrap fájljait tartalmazza
- src mappa -> a felhasznált képeket, betűtípusokat, komponenseket, oldalakat, tesztfájlokat, függvényeket tartalmazza, valamint a weboldal keretének fő .jsx fájljait tartalmazza
- eslint.config.js -> az ESLint konfigurációit tartalmazza
- index.html -> a weboldal HTML oldala
- logoBlack.ico, logoWhite.ico -> a weboldal favicon-jai, annak függvényében változnak, hogy éppen sötét vagy világos témájú a böngésző
- package-lock.json -> a projekt függőségeinek verzióit rögzíti
- package.json -> a projekt alap adatait, függőségeit, és futtatási beállításait tartalmazza
- szerver.html, és server.css -> a szerver html oldala

- vite.config.js -> a Vite konfigurációit tartalmazza



Kép 6 – A frontend mappa

2.3 A program különlegesebb algoritmusai

- A játékban nagyon sok összetett, és egyszerűbb algoritmus található, ezek közül talán az alábbiak a legérdekesebbek.

2.3.1 Különlegesebb algoritmusok a backend-en

2.3.1.1 Chemistry kiszámolása

- A chemistry egy többször is visszatérő játékelem, bármilyen csapatépítésről szóló játékmódról is legyen szó.

- A számítása viszonylag bonyolult, de alapvetően 4 kategória van. A játékos nemzetisége, a liga, amiben játszik, a csapat, aminek a játékosa, és van egy extra is.

- Ez az extra az, ami az icon ritkaságú játékosokat nézi. Erre azért van szükség mert nekik máshogy működik az összhang pontozásuk. Illetve a hero ritkaságú játékosoknak is más a pontozásuk ezek a következőkben értelmet fognak nyerni.

```
function chemFromCountryCount(count) {  
  if (count >= 8) return 3;  
  if (count >= 5) return 2;  
  if (count >= 2) return 1;  
  return 0;  
}
```

Kép 7 - játékosszám feltételek

- Először is létrehozunk egy functiont ami paraméterként kap egy “countot”, ez a feltételeknek megfelelő játékosok száma lesz a későbbiekben. Ebben az esetben az ugyanabból a nemzetből származó játékosok száma

- Következő lépés pedig, hogy szimplán meghatározzuk, hogy mik legyenek a határai, pl.: ha az azonos nemzetből származó játékosok száma nagyobb vagy egyenlő mint 8 akkor az összes játékos, ami megfelel a feltételnek 3 chemistry pontot kap. A return érték mindig a chemistry pontok száma kell legyen

- Ezeket elvégezzük a többi fent felsorolt kategóriánál, ezek így néznek ki:

```
function chemFromLeagueCount(count) {  
  if (count >= 8) return 3;  
  if (count >= 5) return 2;  
  if (count >= 3) return 1;  
  return 0;  
}  
function chemFromClubCount(count) {  
  if (count >= 7) return 3;  
  if (count >= 4) return 2;  
  if (count >= 2) return 1;  
  return 0;  
}  
function chemFromIconCount(count) {  
  if (count >= 6) return 3;  
  if (count >= 4) return 2;  
  if (count >= 2) return 1;  
  return 0;  
}
```

Kép 8 - összes játékoszám feltétel

- Ezután következik annak a leellenőrzése, hogy az adott játékos olyan pozíción van-e amit tud játszani, hiszen csak akkor számolunk nála chemistry pontokat ha ez így van "off-position" játékosok nem adnak az összhanghoz.

```
function inPosition(player) {  
  const slotPos = player.slotPos;  
  const positions = player.player_positions.split(", ");  
  if (!slotPos) return true;  
  
  return positions.includes(slotPos);  
}
```

Kép 9 - inPosition check

- Itt létrejön a function és adunk neki egy “player” paramétert, ami az éppen vizsgált játékos lesz. Lekérjük az éppen adott slot positionjét és ezt eltároljuk egy változóban, aztán egy másik változóban eltároljuk vesszőnként szétválasztva a pozíciókat, amiben a játékosunk tud játszani. Ha nincs slot positionje akkor egy true értéket fog visszaadni.

- A .includes beépített függvénnyel leellenőrizzük, hogy a slotPos tartalmaz-e olyan értéket, mint a positions változó, ez egy true vagy egy false értéket fog visszaadni.

- A következő lépés hogy létrehozuk a calculateChemistry function-t aminek megadunk egy players paramétert, ez később a játékosok listája lesz.

```
function calculateChemistry(players) {  
  const activePlayers = players.filter((p) => p && inPosition(p));  
  let iconCount = 0;  
  const nationCount = {};  
  const leagueCount = {};  
  const clubCount = {};
```

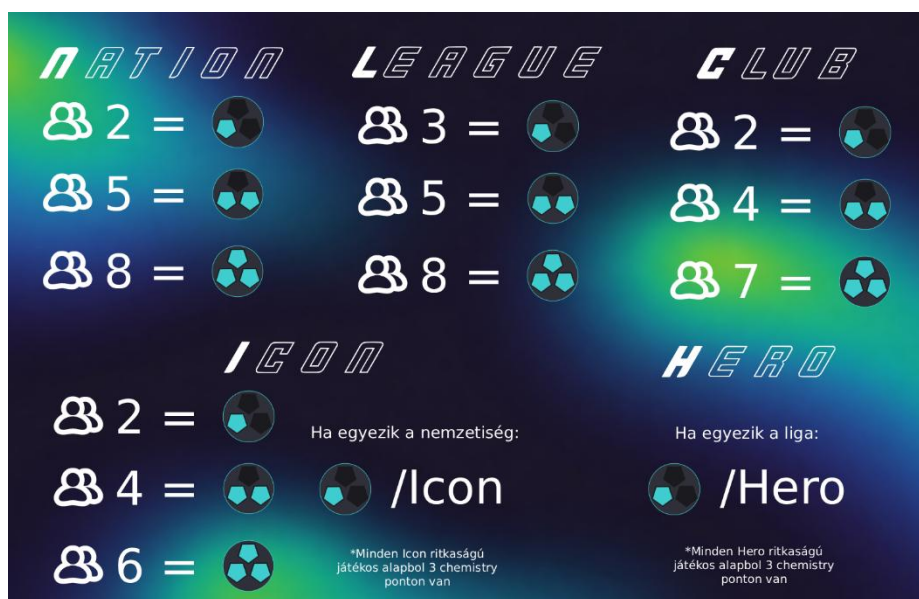
Kép 10 - calculateChemistry function 1

- Létrehozunk egy aktív játékosok array-t, erre a .filter beépített array functiont használjuk. Ha true értéket kap az inPosition function-tól akkor hozzáadja az arrayhez, ha false értéket akkor nem.

- Létrehozunk egy iconCount változót, ami megfogja számolni, hogy hány “icon” ritkaságú játékos van a kezdő 11-ben. Emellett létrehozunk 3 listát, amiben a játékosok nemzetei, ligái, és klubjai lesznek eltárolva.

- Itt következik a játékosok ellenőrzése, itt egy forEach array functionnel egyesével leellenőrizzük az aktív játékosokat az “activePlayers” arrayből

- Segítségül szolgál a következő grafikon az icon és hero súlyok eltalálására



Kép 11 - chemistry grafikon

- így néz ki az ellenőrzés:

```
// számlálás
activePlayers.forEach((p) => {
  if (!nationCount[p.nationality_name]) {
    nationCount[p.nationality_name] = 1;
  } else {
    nationCount[p.nationality_name]++;
  }
  if (p.rarity === "icon") {
    nationCount[p.nationality_name] += 4;
  }
  if (p.rarity === "hero") {
    nationCount[p.nationality_name] += 2;
  }
});
```

Kép 12 - active players forEach

- Ha az adott játékos nemzetisége nincs benne a listában akkor hozzáadja a listához és az értéke 1 lesz. Ellenkező esetben hozzáad még egyet a nemzetiség számához.

- Ha az adott játékos ritkasága icon akkor 4-et ad hozzá ez segít abban, hogy az icon ritkaságú játékos alaptól 3 chemistry ponton legyen és hogy pontosan 1 chemistry pontot adjon minden játékosnak a nemzetben való egyezés.

- Ha az adott játékos ritkasága hero akkor 2ot ad hozzá ez segít abban, hogy az hero ritkaságú játékos alapból 3 chemistry ponton legyen. Ez megtörténik minden egyes kategóriánál.

```
    if (!leagueCount[p.league_name]) {  
      leagueCount[p.league_name] = 1;  
    } else {  
      leagueCount[p.league_name]++;  
    }  
    if (p.rarity === "icon") {  
      leagueCount[p.league_name] += 1;  
    }  
    if (p.rarity === "hero") {  
      leagueCount[p.league_name] += 2;  
    }  
  
    if (!clubCount[p.club_name]) {  
      clubCount[p.club_name] = 1;  
    } else {  
      clubCount[p.club_name]++;  
    }  
    if (p.rarity === "icon") {  
      clubCount[p.club_name] += 1;  
    }  
    if (p.rarity === "hero") {  
      clubCount[p.club_name] += 1;  
    }  
  }  
  if (p.rarity === "icon") {  
    iconCount += 1;  
  }  
});
```

Kép 13 - countok kiszámolása

- A icon és hero hozzáadások mind arra szolgálnak, hogy az ebből a ritkaságból származó játékosok pontosan 3 chemistry ponton legyenek minden esetben.

- Ezután a return-nél egy .map beépített array függvényt használva visszaadjuk a játékosok chemistry értékét.

- Ha nincs játékos akkor a .map egy null értéket fog visszaadni:

```
// chemistry kiszámítás  
return players.map((player) => {  
  if (!player) return null;  
});
```

Kép 14 - player.map 1

- A játékosok, akik benne vannak az arrayben de nincsenek pozíción azoknak csak szimplán vissza lesznek adva az értéke mivel nem kell chemistryt számítani, és kap egy inPosition property-t ami false lesz

```
if (!inPosition(player)) {  
  return {  
    player_id: player.player_id,  
    chemistry: 0,  
    inPosition: false,  
    iconCount: iconCount,  
  };  
}
```

Kép 15 - players.map 2

- A játékosoknál, akik pozíción vannak a következő módon történik a számítás:

```
let chemistry = 0;  
chemistry += chemFromCountryCount(  
  nationCount[player.nationality_name] || 0,  
);  
chemistry += chemFromLeagueCount(leagueCount[player.league_name] || 0);  
chemistry += chemFromClubCount(clubCount[player.club_name] || 0);  
chemistry += chemFromIconCount(iconCount);  
if (chemistry > 3) {  
  chemistry = 3;  
}  
  
return {  
  player_id: player.player_id,  
  chemistry,  
  inPosition: true,  
};  
});
```

Kép 16 - players.map 3

- Meghívjuk az előzőleg már megírt függvényeket, amik kiszámolják minden egyes játékosra, hogy az adott feltétel száma megfelel-e az előre megadott határoknak, ha nem akkor az csak szimplán 0 lesz. Az egy adott játékosra eső chemistry nem lehet nagyobb mint 3 erre szolgál az if feltétel.

- Végül pedig visszaadjuk a játékos adatait, az összhangját és azt, hogy pozíción van.

-Utoljára pedig meghívjuk a draftselectedPlayers11-et, amit eltárolunk a “team” változóban. Emellett létrehozunk egy másik változót “teamWithChemistry” néven, amiben meg eltároljuk a meghívott calculateChemistry function (az előzőleg eltárolt) team változóból visszaadott értékét.

- A response pedig 2 propertyt fog visszaadni: A teamChemistry-t és a players-t. a teamChemistry kiszámítása egy beépített reduce függvény segítségével működik, ami összeadja egyesével a játékosok chemistry értékét, kezdőértéke nulla

```
const team = draftselectedPlayers11;

const teamWithChemistry = calculateChemistry(team);

response.status(200).json({
  teamChemistry: teamWithChemistry.reduce((sum, p) => sum + p.chemistry, 0),
  players: teamWithChemistry,
});
} catch (error) {
  console.log("GET /api/draft/chemistry error:", error);
  response.status(500).json({ message: "Internal server error" });
}
});
```

Kép 17 - response

2.3.1.2 Rating kiszámolása

- A játék során számos helyen szükségünk van a csapat rating-jére, azaz a csapat “értékelésére”.

- Matematikailag ez a következőképpen épül fel:

- Összeadjuk a kezdő 11-nek a rating-jeit
- Összeadjuk a cserejátékosok rating-jeit
- Átlagot vonunk a cserejátékosok rating-jeiből
- Összeadjuk a kezdő 11 ratingjeinek összegét a cserejátékosok átlagával
- Ezt elosztjuk 12-vel
- Kerekítünk

- Ez a fajta megoldás azért jó, mivel a cserejátékosok is valamennyire számítanak, ezért a felhasználónak arra is kell törekednie, hogy oda is minél jobb kártyákat rakjon.

- Erre a `/api/draft/rating` végpontunk szolgál.

- Először is külön tömbbe mentjük a kezdő 11, és a cserejátékosokat.

```
const starting11players = await draftselectedPlayers11;  
const subplayers = await draftselectedPlayersSubs;
```

Kép 18 – a játékosok elmentése egy tömbbe

- Ezek után megnézzük, hogy egyáltalán van-e játékos a kezdő 11-be, és ha nincs, akkor 0 lesz a rating.

```
if (starting11players.length === 0) {  
  return response.json({ rating: 0 });  
}
```

Kép 19 – kezdő 11 hosszának ellenőrzése

- Ezután a kezdő 11-ből, és a cserepadból is kisorsoljuk a rating-eket, és külön tömbökben tároljuk.

```
const starting11ratings = starting11players.map((p) => Number(p.overall));  
const subratings = subplayers.map((p) => Number(p.overall));
```

Kép 20 – rating-ek “kiszordítása”

- Innentől kezdve pedig azt csináljuk, amit korábban már említettünk.

```
let starting11ratingsSum = 0;  
for (let i = 0; i < starting11ratings.length; i++) {  
  starting11ratingsSum += starting11ratings[i];  
}  
  
let subratingsSum = 0;  
for (let i = 0; i < subratings.length; i++) {  
  subratingsSum += subratings[i];  
}  
const subavg = Math.round(subratingsSum / 7);  
  
const correctedSum = starting11ratingsSum + subavg;  
  
const correctedSumAvg = correctedSum / 12;  
  
const finalRating = Math.round(correctedSumAvg);
```

Kép 21 – rating megkapása számítások után

- Ezek után már csak response-t kell küldeni.

```
response.status(200).json({  
  rating: finalRating,  
});
```

Kép 22 – rating response

2.3.1.3 5 random játékos legenerálása pozíciónként

- A draft játékmód arról szól, hogy random játékosok közül lehessen választani. Erre a célra készült a /api/draft/random/:pos végpont.

- Paraméterként megkapja ez a végpont a pozíciót, amire klikkelt a felhasználó. Ezáltal csak olyan játékost fogunk kapni, amely tud játszani abban az adott pozícióban.

- Emellett beolvassuk a players.json fájlt, innen lesznek majd a random játékosok.

```
const pos = request.params.pos;  
const data = await readJsonFile(  
  path.join(__dirname, "../files/players.json"),  
);
```

Kép 23 – pos és data a random játékos generálásnál

- Ezután megnézzük, hogy milyen játékosokat választott már a felhasználó, játékos azonosító alapján. Ez később azért fontos, hogy ne lehessen ugyanazt a játékost újra és újra felhasználni.

```
const selectedIds = draftselectedPlayers.map((p) => p.player_id);
```

Kép 24 – kiválasztott játékosok id-ja a draft-ban

- Mivel a cserepadon nem fix pozíciók vannak, hanem külön védő, támadó, vagy adott esetben bármilyen, ezért külön csoportokat hozunk létre, hogy amennyiben a felhasználó a cserepadra szeretne játékost tenni, abban az esetben az összes pozíciót vegye figyelembe, ami abba az adott csoportba tartozik.

```
const posGroups = {  
  DEF: ["LB", "CB", "RB"],  
  MID: ["LM", "RM", "CM", "CDM", "CAM"],  
  ATT: ["LW", "RW", "ST"],  
  ANY: [  
    "LW",  
    "RW",  
    "ST",  
    "LM",  
    "RM",  
    "CDM",  
    "CM",  
    "CAM",  
    "RB",  
    "CB",  
    "LB",  
    "GK",  
  ],  
};
```

Kép 25 – pozíció csoportok

- Megnézzük, hogy a paraméterként kapott pos szerepel-e a pozíció csoportokban. Ha szerepel, akkor az összes pozíciót figyelembe veszi majd a random játékos generálás, ha pedig nem, akkor csak azt az egy adott pozíciót, amire klikkelt a felhasználó.

```
const getPositions = (pos) => {  
  return posGroups[pos] || [pos];  
};
```

Kép 26 – getPositions függvény

- Mivel az egész draft a szerencsén múlik, viszont azt sem szeretnénk, hogy túl nehéz legyen jó draft-ot építeni, ezért rating range-eket csináltunk, ami azt teszi lehetővé, hogy súlyozás segítségével általában jó, vagy kiváló kártyákat kapunk a választásba. Ennek köszönhetően kiszámíthatatlan, mégis élvezetes tud maradni a játékmenet.

- Ez úgy működik, hogy generáltunk egy random számot 0 és 9 között, és mindig az a szám alapján kapjuk meg a generálás minimum rating-jét, és maximum rating-jét.

- A százalékok az alábbiak:

- 75-80 -> 10%
- 81-84 -> 20%
- 85-87 -> 40%
- 88-99 -> 30%

```

const getRatingRange = () => {
  const randomRatingRange = Math.floor(Math.random() * 10);

  if (randomRatingRange === 0) {
    return { min: 75, max: 80 };
  }

  if (randomRatingRange === 1 || randomRatingRange === 2) {
    return { min: 81, max: 84 };
  }

  if (
    randomRatingRange === 3 ||
    randomRatingRange === 4 ||
    randomRatingRange === 5 ||
    randomRatingRange === 6
  ) {
    return { min: 85, max: 87 };
  }

  return { min: 88, max: 99 };
};

```

Kép 27 – rating range-ek a draft-ban

- Ezután jöhet már a tényleges random játékos generálás.
- Addig fog futni, amíg nincs meg a szükséges elemszám, az 5, vagy amíg el nem fogynak az elemek.
- Random generálunk egy index-et, ami szerepel a játékosok tömbjében.
- Megnézzük, hogy a kapott játékos játszhat-e a választott pozícióban.
- Ezután pedig bekerül a randomPlayers tömbbe, amennyiben még nem szerepel az akkori randomPlayers tömbben, beleesik a rating range-be, megfelel a pozíciója, vagy nincs már a csapatban.
- Ha megfelel minden feltételnek, akkor hozzáadjuk a kiválasztható játékosokhoz, és hozzáadjuk a játékos azonosítóját a usedPlayerIds tömbhöz, hogy ne lehessen kétszer ugyanaz a játékos a kiválasztásban.

- Végül pedig adunk egy response-t, amiben meghívjuk a random generálás függvényét, aminek átadjuk a players.json-t és azt, hogy 5 játékost szeretnénk.

```
const randomPick = (arr, count) => {
  const randomPlayers = [];
  const tombIndexek = [];
  const usedPlayerIds = [];
  const allowedPos = getPositions(pos);
  const { min, max } = getRatingRange();

  while (randomPlayers.length < count && tombIndexek.length < arr.length) {
    const index = Math.floor(Math.random() * arr.length);
    const player = arr[index];
    const playerPositions = player.player_positions.split(", ");

    const validPosition = allowedPos.some((position) =>
      playerPositions.includes(position),
    );

    if (
      !tombIndexek.includes(index) &&
      player.overall >= min &&
      player.overall <= max &&
      validPosition &&
      !usedPlayerIds.includes(player.player_id) &&
      !selectedIds.includes(player.player_id)
    ) {
      tombIndexek.push(index);
      usedPlayerIds.push(player.player_id);
      randomPlayers.push(player);
    } else if (!tombIndexek.includes(index)) {
      tombIndexek.push(index);
    }
  }

  return randomPlayers;
};

return response.status(200).json({ randomPlayers: randomPick(data, 5) });
```

Kép 28 – 5 random játékos generálása

2.3.1.4 Pakli generálása

- A pakli generálás egy súlyozott logikán alapul, a súlyok minden paklinál mások és előre megvannak határozva egy sql táblában. Egy get végponttal és sql query-k felhasználásával. A következőkben megláthatjuk, hogy ezek a táblák hogy épülnek fel:

id	packName	packPrice	packDesign	playerCount	playerQuality	packWeightID	1
1	Bronze Pack	750	bronze	12	bronze		1
2	Silver Pack	3000	silver	12	silver		2
3	Gold Pack	7500	gold	12	gold/flashback/scream/toty/icon/hero		3
4	Jumbo Gold Pack	12000	gold	24	gold/flashback/scream/toty/icon/hero		4
5	Mixed Players Pack	6000	special	12	bronze/silver/gold		5
6	Mega Pack	35000	special	30	gold/flashback/scream/toty/icon/hero		6
7	80+×10 Players Pack	20000	special	10	gold/flashback/scream/toty/icon/hero		7
8	82+×10 Players Pack	25000	special	10	gold/flashback/scream/toty/icon/hero		8
9	85+×4 Players Pack	30000	special	4	gold/flashback/scream/toty/icon/hero		9
10	Icon Pack	100000	special	1	icon		10
11	90+ Icon Pack	200000	special	1	icon		11
12	SuloFUT Flashback Pack	70000	flashback	12	gold/flashback/scream/toty/icon/hero		12
13	SuloFUT Scream Pack	50000	scream	12	gold/flashback/scream/toty/icon/hero		13
14	Hero Pack	60000	special	1	hero		14
15	87+ Hero Pack	20000	special	1	hero		15
16	Campaign Mix Pack	80000	special	1	flashback/scream/toty/icon/hero		16
17	88+ Hero or Icon Pack	80000	special	1	icon/hero		17
18	TOTY Grind Pack	50000	toty	12	gold/flashback/scream/toty/icon/hero		18
19	TOTY Pack	200000	toty	1	toty		19
20	Welcome Pack	0	special	24	bronze/silver/gold		20

Kép 30 - pack tábla

id	bronzeWeight	silverWeight	goldWeight	flashbackWeight	screamWeight	totyWeight	iconWeight	heroWeight
1	100	0	0	0	0	0	0	0
2	0	100	0	0	0	0	0	0
3	0	0	94	2	2	0.5	0.5	1
4	0	0	97	1	1	0.3	0.3	0.4
5	33	33	33	0.3	0.3	0.1	0.1	0.2
6	0	0	98	0.7	0.7	0.1	0.1	0.4
7	0	0	94	2	2	0.5	0.5	1
8	0	0	98	0.7	0.7	0.1	0.1	0.4
9	0	0	80	6	6	2	2	4
10	0	0	0	0	0	0	100	0
11	0	0	0	0	0	0	100	0
12	0	0	94	3	1	0.5	0.5	1
13	0	0	94	1	3	0.5	0.5	1
14	0	0	0	0	0	0	0	100
15	0	0	0	0	0	0	0	100
16	0	0	0	33	33	33	0	0
17	0	0	0	0	0	0	30	70
18	0	0	94	1.5	1.5	1.5	0.5	1
19	0	0	0	0	0	100	0	0
20	33	33	33	0	0	0	0	0

Kép 29 - packWeight tábla

- Itt megtörténik a paklira vonatkozó sql táblában tárolt adatok lekérdezése a storeQueries.js fájlban. Itt lekérjük a pack játékos számát és a packben megtalálható játékosok ritkaságát. Ezen kívül a másik lekérdezésben pedig az összes weight adatját:

```
/** Pack játékosszámának és játékos rarity-jeinl lekérése id szerint
async function getPackDetails(id) {
  const query = "SELECT playerCount, playerQuality FROM packs WHERE id = ?";
  try {
    const [rows] = await pool.execute(query, [id]);
    return rows[0];
  } catch (error) {
    throw error;
  }
}

/** Pack weight id szerint
async function getAllWeightData(id) {
  const query =
    "SELECT packWeights.bronzeWeight, packWeights.silverWeight, packWeights.goldWeight, packWeights.flashbackWeight, packWeights.screamWeight, packWeights.totyWeight, packWeights.iconWeight, packWeights.heroWeight FROM packWeights JOIN packs ON packWeights.id = packs.packWeightID WHERE packs.id = ?";
  try {
    const [rows] = await pool.execute(query, [id]);
    return rows[0];
  } catch (error) {
    throw error;
  }
}
```

Kép 31 - lekérdezések

- Először is a get módszerünk egy paraméteres módszer, ami a paklinak az azonosítóját várja paraméterként.

```
///! Pack player generálás
router.get("/generatePack/:id", async (request, response) => {
  try {
    const id = request.params.id;
    const packWeights = await storeQueries.getAllWeightData(id);
    const packData = await storeQueries.getPackDetails(id);
    const playerCount = packData.playerCount;
    const data = await readJsonFile(
      path.join(__dirname, "./files/players.json"),
    );
    const conds = {
      7: 80,
      8: 82,
      9: 85,
      11: 90,
      15: 87,
      17: 88,
    };
  }
});
```

Kép 32 - get módszer inicializálása

- A fent látható képen elég sok minden történik szóval nézzük meg egyesével.
- Először is lekérjük a paraméterben megadott id-t és ezt eltároljuk egy változóban.
- Aztán csinálunk két változót a két query-nk eredményének, a playerCount-ot egy külön változóban is eltároljuk.
- Beolvassuk a játékosokhoz tartozó Json fájl értékeit, amit szintén eltárolunk egy változóban.
- Végül pedig létrehozunk egy “conds” nevezetű listát, amiben bizonyos pack id-k vannak property-ként. Ezek olyan packek amikben van minimum rating megszorítás. Ezek a minimum értékek pedig a property-k értékei.

- Következő lépés pedig a randompack function létrehozása, ami 3 paramétert fog kapni. “arr” ez a játékosokat tartalmazó tömb lesz, “count”, ami a query-ből kinyert játékos szám lesz és “condition”, ami az előzőleg használt “conds” lista lesz

- A function-on belül kezdésből létrehozunk egy randomjatekosok tömböt és egy tombIndexek tömböt. Illetve létrehozunk egy playerQuality array-t amiben a packdata.playerQuality-t könnyebben feldolgozhatóra formáljuk beépített function-okkal. Kezdve a .split függvénnyel, amivel “/” jelek mentén elválasztjuk a ritkaságokat, aztán a .map array és a .trim string függvénnyel megszüntetjük a whitespace-eket.

```
function randompack(arr, count, condition) {  
  const randomjatekosok = [];  
  const tombIndexek = [];  
  const playerQuality = packData.playerQuality  
    .split("/")  
    .map((q) => q.trim());
```

Kép 33 - randompack function inicializálása

- Ezt követően csinálunk egy while ciklust ami addig aktív amíg a randomjatekosok.length kisebb, mint a paraméterben meghatározott count és amíg a tombIndexek.length kisebb mint a paraméterben meghatározott arraynek a hossza, ez a while majdnem a function végéig tart.

```

const rarityCount = {};
for (let i = 0; i < arr.length; i++) {
  if (
    !tombIndexek.includes(i) &&
    playerQuality.includes(arr[i].rarity)
  ) {
    const rarity = arr[i].rarity;
    rarityCount[rarity] = (rarityCount[rarity] || 0) + 1;
  }
}

const rarityWeights = [];
let totalWeight = 0;

```

Kép 34 - rarityCount

- A fent látható képen létrehozunk egy rarityCount listát, amit később a ritkaságok számával fogunk feltölteni. Utána egy for ciklus segítségével megnézzük egy beépített függvény segítségével, hogy a jelenlegi index volt-e már használva és hogy a “playerQuality” tartalmazza-e a vizsgált játékos ritkaságát. Ha igen akkor létrehozunk egy változót, ami egyenlő a vizsgált játékos ritkaságával
- Ezután, ha a “rarityCount”ban az a ritkaság még nem szerepel akkor undefined lesz és ezért a vagy feltétel miatt 0 lesz az értéke a +1 pedig arra szolgál, hogy így 1 legyen, ha létezik már akkor az adott számát használja és hozzáad 1et
- A következőkben létrehozunk egy “rarityWeights” tömböt és egy “totalWeight” változót, aminek az értéke 0. Folytatjuk egy “for of” ciklussal, ami végig járja az összes key-ét a rarityCount listának, ebben létrehozunk egy weight változót is, aminek a kezdő értéke 0.
- Következnek az if feltételek, amik azt vizsgálják, hogy a bizonyos key-nek a rarity-je benne van e a packweight listában, ha benne van akkor lekéri a weight value-ját (vagy ha nincs akkor alapértelmezetten 1) és egyenlővé teszi a weight változóval.

- Ez az ellenőrzés megtörténik minden egyes ritkaságnál és utána minden egyes igaz if feltételnél a “weight” változó értéke hozzáadódik a “totalWeight” változóhoz, ezután a rarityWeights listához adjuk a rarity-t és a weight-et a .push beépített array functionnel.

```
const rarityWeights = [];  
let totalWeight = 0;  
  
for (const rarity of Object.keys(rarityCount)) {  
  let weight = 0;  
  
  if (rarity.includes("bronze")) weight = packWeights.bronzeWeight || 1;  
  else if (rarity.includes("silver"))  
    weight = packWeights.silverWeight || 1;  
  else if (rarity.includes("gold"))  
    weight = packWeights.goldWeight || 1;  
  else if (rarity.includes("flashback"))  
    weight = packWeights.flashbackWeight || 1;  
  else if (rarity.includes("scream"))  
    weight = packWeights.screamWeight || 1;  
  else if (rarity.includes("toty"))  
    weight = packWeights.totyWeight || 1;  
  else if (rarity.includes("icon"))  
    weight = packWeights.iconWeight || 1;  
  else if (rarity.includes("hero"))  
    weight = packWeights.heroWeight || 1;  
  
  totalWeight += weight;  
  rarityWeights.push({ rarity, weight });  
}
```

Kép 35 - rarityCount ellenőrzése

- Létrehozunk ezután egy “random” változót, aminek az értéke egy véletlenszerű szám, amit a Math.random() statikus metódussal generálunk megszorozva a “totalWeight” változó értékével. Illetve létrehozunk egy “selectedRarity” nevű változót, amelynek alapértelmezett értéke null

- Egy for ciklus segítségével pedig kivonogatjuk egyesével az adott rarity-k weightjeit amíg el nem jutunk nulláig, ha ez teljesül akkor megvan a selectedRarity-nk, ezért hozzáadjuk azt a rarity-t aminél megállt a kivonás.

```
let random = Math.random() * totalWeight;
let selectedRarity = null;

for (let i = 0; i < rarityWeights.length && !selectedRarity; i++) {
  random -= rarityWeights[i].weight;
  if (random <= 0) {
    selectedRarity = rarityWeights[i].rarity;
  }
}
```

Kép 36 - selectedRarity

- Hogyha megvan a selectedRarity akkor pedig létrehozunk egy validPlayers tömböt és elindítunk egy for ciklust, amiben 3 feltételnek kell megfeleljen a játékos. 1. Nem tartalmazza már az indexét a tombIndexek, 2. A ritkasága egyenlő-e a kiválasztott ritkasággal, 3. Az értékelése nagyobb-e, mint a condition. Ha ezeknek mind megfelel akkor a validPlayers tömbhöz lesz hozzáadva az indexével együtt.

```
if (selectedRarity) {
  const validPlayers = [];
  for (let i = 0; i < arr.length; i++) {
    if (
      !tombIndexek.includes(i) &&
      arr[i].rarity === selectedRarity &&
      arr[i].overall >= condition
    ) {
      validPlayers.push({ index: i, player: arr[i] });
    }
  }
}
```

Kép 37 - validPlayers

- Következő lépés, hogy egy if feltételben meghatározzuk hogyha a validPlayers tömb hossza nagyobb mint 0, akkor létrehozunk egy randomIndex változót. Ennek az értéke egy egészre kerekített random szám, amit a validPlayers hosszával szorzunk meg. Utána ezt felhasználjuk amikor létrehozunk egy változót "selected" néven a következő formában: a validPlayers tömb randomIndex-edik helyén lesz a játékosunk így ezután pusholjuk a tömbindexekbe a kiválasztott játékos indexét és a randomjatekosok tömbbe pedig a kiválasztott játékost.

```
if (validPlayers.length > 0) {  
  const randomIndex = Math.floor(Math.random() * validPlayers.length);  
  const selected = validPlayers[randomIndex];  
  tombIndexek.push(selected.index);  
  randomjatekosok.push(selected.player);  
}  
}
```

Kép 38 - kiválasztott játékosok pusholása

- Végző soron pedig a .sort beépített függvénnyel elrendezzük értékelés szerint csökkenő sorrendbe a random játékosokat és return-öljük

- Létrehozunk egy condition változót, ami a conds[id]-t várja ez az id a pack id-je amit paraméterként elkérünk, ha egyezik akkor az lesz a condition értéke, ha nem akkor 0

- Itt már csak egy response marad, amiben meghívjuk a randompack függvényt és átadjuk neki paraméterként a data-t, a playerCount-ot és a conditiont

```
randomjatekosok.sort((a, b) => b.overall - a.overall);  
  
return randomjatekosok;  
}  
const condition = conds[id] || 0;  
  
response  
  .status(200)  
  .json({ randomjatekosok: randompack(data, playerCount, condition) });  
} catch (error) {  
  console.log("GET /api/store/generatePack/:id error:", error);  
  response.status(500).json({ message: "Internal server error" });  
}  
});
```

Kép 39 - responseban a kiválasztott játékosok hozzáadása

- így néz ki a teljes végpont:

```
#!/ Pack player generálás
router.get("/generatePack/:id", async (request, response) => {
  try {
    const id = request.params.id;
    const packWeights = await storeQueries.getAllWeightData(id);
    const packData = await storeQueries.getPackDetails(id);
    const playerCount = packData.playerCount;
    const data = await readJsonFile(
      path.join(__dirname, "../files/players.json"),
    );
    const conds = {
      7: 80,
      8: 82,
      9: 85,
      11: 90,
      15: 87,
      17: 88,
    };

    function randompack(arr, count, condition) {
      const randomjatekosok = [];
      const tombIndexek = [];
      const playerQuality = packData.playerQuality
        .split("/")
        .map((q) => q.trim());

      while (
        randomjatekosok.length < count &&
        tombIndexek.length < arr.length
      ) {
        const rarityCount = {};
        for (let i = 0; i < arr.length; i++) {
          if (
            !tombIndexek.includes(i) &&
            playerQuality.includes(arr[i].rarity)
          ) {
            const rarity = arr[i].rarity;
            rarityCount[rarity] = (rarityCount[rarity] || 0) + 1;
          }
        }

        const rarityWeights = [];
        let totalWeight = 0;

        for (const rarity of Object.keys(rarityCount)) {
          let weight = 0;

          if (rarity.includes("bronze")) weight = packWeights.bronzeWeight || 1;
          else if (rarity.includes("silver"))
            weight = packWeights.silverWeight || 1;
          else if (rarity.includes("gold"))
            weight = packWeights.goldWeight || 1;
          else if (rarity.includes("flashback"))
            weight = packWeights.flashbackWeight || 1;
          else if (rarity.includes("scream"))
            weight = packWeights.screamWeight || 1;
          else if (rarity.includes("toty"))
            weight = packWeights.totyWeight || 1;
          else if (rarity.includes("icon"))
            weight = packWeights.iconWeight || 1;
          else if (rarity.includes("hero"))
            weight = packWeights.heroWeight || 1;

          totalWeight += weight;
          rarityWeights.push({ rarity, weight });
        }

        let random = Math.random() * totalWeight;
        let selectedRarity = null;

        for (let i = 0; i < rarityWeights.length && !selectedRarity; i++) {
          random -= rarityWeights[i].weight;
          if (random <= 0) {
            selectedRarity = rarityWeights[i].rarity;
          }
        }

        if (selectedRarity) {
          const validPlayers = [];
          for (let i = 0; i < arr.length; i++) {
            if (
              !tombIndexek.includes(i) &&
              arr[i].rarity === selectedRarity &&
              arr[i].overall >= condition
            ) {
              validPlayers.push({ index: i, player: arr[i] });
            }
          }

          if (validPlayers.length > 0) {
            const randomIndex = Math.floor(Math.random() * validPlayers.length);
            const selected = validPlayers[randomIndex];
            tombIndexek.push(selected.index);
            randomjatekosok.push(selected.player);
          }
        }
      }
      randomjatekosok.sort((a, b) => b.overall - a.overall);
      return randomjatekosok;
    }

    const condition = conds[id] || 0;

    response
      .status(200)
      .json({ randomjatekosok: randompack(data, playerCount, condition) });
  } catch (error) {
    console.log("GET /api/store/generatePack/:id error:", error);
    response.status(500).json({ message: "Internal server error" });
  }
});
```

2.3.2 Különlegesebb algoritmusok a frontend-en

2.3.2.1 Védett útvonalak

- Mivel a játékban minden módosítás stb. eltárolásához felhasználó szükséges, ezért felhasználói profil kötelező a játék használatához. Ennek megoldására azt szeretnénk volna, hogy amíg a felhasználó nincs még bejelentkezve, addig ne férjen hozzá a játékhoz, csak a bejelentkezés és a regisztrációs felülethez.
- Ezt a `ProtectedRoute.jsx` segítségével valósítottuk meg.
- Minden Route, amit védeni szeretnénk volna, ezzel van védve.

```
<Route element={<ProtectedRoute />}>
  <Route path="/" element={<MainPage />} />
  <Route path="/leaderboard" element={<LeaderboardPage />} />
  <Route path="/objectives" element={<ObjectivesPage />} />
  <Route path="/draft" element={<DraftPage />} />
  <Route path="/store" element={<StorePage />} />
  <Route path="/sbc" element={<SBCPage />} />
  <Route path="/settings" element={<SettingsPage />} />
  <Route path="/club" element={<ClubPage />} />
  <Route path="/squad" element={<SquadPage />} />
</Route>
```

Kép 41 – A `ProtectedRoute` védi az oldalakat

- A `ProtectedRoute` komponens elején egy `useState` hook-ot hoztunk létre, a login állapotának eltárolására.

```
const [isLoggedIn, setIsLoggedIn] = useState(null);
```

Kép 42 – `isLoggedIn` `useState`

- Ezután useEffect hook-ban létrehoztunk egy aszinkron function-t, ami a komponens meghívásakor kerül meghívásra. (Mivel üres a dependency tömb, ezért csak egyszer fut le.) Ez ellenőrzi egy API végponton keresztül, hogy be van-e jelentkezve a felhasználó. Amennyiben nincs, és error-t kap a végpont meghívásakor, akkor false-ra állítja a korábban létrehozott állapotot.

```
useEffect(() => {
  const checkLogin = async () => {
    try {
      const response = await fetch("http://127.0.0.1:3000/api/users/me", {
        credentials: "include",
      });
      setIsLoggedIn(response.ok);
    } catch {
      setIsLoggedIn(false);
    }
  };
  checkLogin();
}, []);
```

Kép 43 – checkLogin function

Amennyiben viszont megfelelő a response, abban az esetben true-ra állítja az állapotot.

- Amíg az ellenőrzés nem fejeződik be, addig nem return-ölünk semmit, hogy a komponens ne rendereljen.

- Amennyiben az isLoggedIn állapota false-ra változik, a felhasználót átirányítja a bejelentkezés felületére, ha pedig true-ra, akkor render-eli a komponens a védett route-hoz tartozó gyerek komponenseket.

```
///! Átirányítás a login-hoz
if (!isLoggedIn) {
  return <Navigate to="/login" replace />;
}

///! Oldal renderelése ha be van jelentkezve
return <Outlet />;
```

Kép 44 – átirányítás a bejelentkezéshez, vagy a komponensek renderelése

2.3.2.2 Drag-elés

- A játékban számos helyen előfordul, hogy meg szeretnénk cserélni két játékost a csapatunkban. Ehhez készítettünk egy custom hook-ot, ami a drag.js-ben található. Lényegében ez a drag-and-drop azt teszi lehetővé, hogy egy HTML elemet meg tudjunk fogni, és ha egy, már a csapatban szereplő játékos felé húzzuk, és elengedjük a húzott játékost, megcserélődik a kettő játékos.

- A komponens három állapotot kezel useState segítségével. Az isDragging jelzi, hogy éppen történik-e drag, a dragKey a húzott elem azonosítója, a dragPos pedig a kurzor/ujj(mobilon) pozíciója.

```
const [isDragging, setIsDragging] = useState(false);  
const [dragKey, setDragKey] = useState(null);  
const [dragPos, setDragPos] = useState({ x: 0, y: 0 });
```

Kép 45 – Drag állapotok

- A getPoint függvény segítségével egyesítjük az egér és a touch eseményeket. Ennek segítségével tudjuk egységesen kezelni a koordinátákat, mivel egéرنél pl. az X koordinátát az e.clientX segítségével tudjuk lekérni, mobilon viszont ugyanezt az e.touches[0].clientX segítségével tehetjük meg.

```
const getPoint = (e) => (e.touches ? e.touches[0] : e);
```

Kép 46 – getPoint függvény

- A startDrag függvény felel a drag inicializálásáért.

- Csak bal egérgombbal van engedélyezve a drag.

- A preventDefault() tiltja pl. a kijelölést, hogy ne zavarjon be.

- Ezeken kívül eltárolja a húzott elem/játékos azonosítóját, valamint a kurzor/ujj pozícióját.

```
const startDrag = (e, key) => {  
  if (e.type === "mousedown" && e.button !== 0) return;  
  e.preventDefault();  
  const point = getPoint(e);  
  setDragKey(key);  
  setIsDragging(true);  
  setDragPos({ x: point.clientX, y: point.clientY });  
};
```

Kép 47 – startDrag függvény

- Maga a Drag esemény közben a useEffect hook biztosítja, hogy az események csak drag során legyenek figyelve.
- A handleMove folyamatosan frissíti a kurzor/ujj pozícióját. Ez teszi lehetővé a dragPreview-t is, amely segítségével szépen húzhatjuk jobbra-balra a játékost.

```
const handleMove = (e) => {  
  const point = getPoint(e);  
  setDragPos({ x: point.clientX, y: point.clientY });  
};
```

Kép 48 – handleMove függvény

- Drag befejezésekor a handleUp függvény kerül meghívásra.
- Ennek feladata, hogy meghatározza, hogy melyik DOM elem alatt történt a drop. A slot-ot ahova le szeretnénk dobni pedig a kurzor alatti [data-slotkey] alapján azonosítja (amennyiben van). Ezután pedig eltárolja egy változóban a cél data-slotkey-t.
- False-ra változtatja az isDragging-et, és a dragKey-t null-ra, ezzel “reset-eli” a drag-et.

- Ezek után amennyiben minden sikeres volt (semmi sem null), meghívja a `handleSwapPlayers` függvényt, aminek átadja a húzott, és a cél elem azonosítóit. Ennek segítségével valósul meg valójában a két játékos megcserélése mind a frontend-en, mind a backend-en.

```
const handleUp = async (e) => {
  const point = getPoint(e);
  const element = document.elementFromPoint(point.clientX, point.clientY);
  const slot = element?.closest("[data-slotkey]");
  const to = slot?.getAttribute("data-slotkey");
  setIsDragging(false);
  setDragKey(null);

  if (!to || dragKey === null || !assignedPlayers[to]) return;
  await handleSwapPlayers(dragKey, to);
};
```

Kép 49 – `handleUp` függvény

- Ezek után kezeltük az event listener-eket, mivel nem szeretnénk, hogy megakadjon a drag, ha a kurzor kilóg a húzott elemből, vagy túl gyorsan mozog. Ezért magára a window-ra tettünk event listener-eket, és a `useEffect` végén a `return`-be eltávolítjuk ezeket, hogy ne legyenek duplikált események.

```
window.addEventListener("mousemove", handleMove);
window.addEventListener("mouseup", handleUp);
window.addEventListener("touchmove", handleMove);
window.addEventListener("touchend", handleUp);
return () => {
  window.removeEventListener("mousemove", handleMove);
  window.removeEventListener("mouseup", handleUp);
  window.removeEventListener("touchmove", handleMove);
  window.removeEventListener("touchend", handleUp);
};
```

Kép 50 – Event listener-ek kezelése a drag-ben

- Maga a drag újrafut, ha a drag állapot változik, ha a húzott elem változik, vagy amennyiben a külső adatok változnak, mivel szerepelnek a `useEffect` dependency tömbjében. Ez biztosítja, hogy mindig aktuális állapottal dolgozzon a drag.

- Végül pedig visszatérési értéként átadjuk azt, hogy elkezdődött-e a drag, az isDragging állapotát, a húzott elem azonosítóját, és a kurzor/ujj pozícióját. Ezek segítségével szép UI-t lehet adni a drag-nek, valamint meg lehet valósítani a tényleges megcserélést.

```
}, [isDragging, dragKey, handleSwapPlayers, assignedPlayers]);
```

Kép 51 – Drag komponens visszatérési értékei

```
return { isDragging, dragKey, dragPos, startDrag };
```

Kép 52 – drag useEffect dependency-k

2.3.2.3 Formáció legenerálása

- A játék során sok helyen szükségünk van arra, hogy megjelenjen egy adott formáció, és arra, hogy szabadon tudjunk játékost kiválasztani stb.

- Erre készítettük a Gamelayout.jsx-et, ami szintén teljes mértékben egy újrahasználatos komponens.

- A komponens működéséhez elég sok props-ra van szükség, amit megkap a meghíváskor.

```
function Gamelayout({
  gameLayout,
  assignedPlayers,
  isDragging,
  dragKey,
  handlePosClick,
  startDrag,
  chemImg,
  playerChemMap,
  displayedPosition,
  allowReplace,
}) {
```

Kép 53 – Gamelayout props-ok

- A komponens a gameLayout tömbön iterál, ami minden pozícióhoz tartalmaz position-öket, és pozícióneveket.

- Minden elemhez egy pozícionált slot kerül renderelésre, majd később a pozíció felírata is. Megkapja a slot a data-slotkey-t, ami később kulcsfontosságú lesz a drag-nél, megkapja id-nek a pozíció típusát (pl. GK,

CAM, stb.), valamint left, és top pozícionálással megkapja a pozícióját is az oldalon.

- Ezen kívül még egy klikk eseményt is kap, ami meghívja a handlePosClick() külső függvényt (ami majd később meghívja a játékos kiválasztó modal-t), kivéve, ha van már játékos abban az adott pozícióban, és nincs engedélyezve a játékos cserélése (draft játékmódban).

```
{gameLayout.map((p, i) => (  
  <div  
    key={i}  
    className="pos"  
    data-slotkey={i}  
    style={{ left: p.x + "%", top: p.y + "%" }}  
    id={p.pos}  
    onClick={() => {  
      if (!allowReplace && assignedPlayers[i]) return;  
      handlePosClick(i, p.pos);  
    }}  
  >  
)
```

Kép 55 – slot-ok generálása

```
{gameLayout.map((p, i) => (  
  <p  
    key={i}  
    className="posText"  
    style={{ left: p.x + "%", top: `calc(${p.y}% + 6%)` }}  
  >  
    {p.pos}  
  </p>  
)
```

Kép 54 – pozíció kiírása a slot alá

- Amennyiben van a slot-hoz játékos hozzárendelve, akkor megjelenik a játékos is, egy külön wrapper div-ben.
- Ez a wrapper megkapja a drag indítását egérrel és touch-al.
- A játékos kártyájának tényleges megjelenését viszont már a PlayerCard komponens végzi.

```
{assignedPlayers[i] && (
  <div
    onMouseDown={(e) => startDrag(e, i)}
    onTouchStart={(e) => startDrag(e, i)}
    className={`cardSlot ${
      isDragging && dragKey === i ? "dragSource" : ""
    }`}
  >
    <PlayerCard
      player={assignedPlayers[i]}
      isDragging={isDragging}
      isSource={dragKey === i}
      chemImg={chemImg}
      playerChemMap={playerChemMap}
      displayedPosition={displayedPosition}
      slotPos={p.pos}
      isModal={false}
      onClick={() => {
        if (!allowReplace && assignedPlayers[i]) return;
        handlePosClick(i, p.pos);
      }}
    />
  </div>
)}
```

Kép 56 – játékosok hozzáadása a formációhoz

2.3.2.4 Kártya legenerálása

- A játékban talán a legtöbbet használt komponens a játékos kártyájának legenerálásáért szolgáló PlayerCard.jsx. Ez felel a kártyák teljes vizuális megjelenéséért. A komponens különböző adatok pl. pozíció, klub, nemzetiség, rarity, stb. alapján dinamikusan rendereli a UI-t.
- A komponens memo-val van optimalizálva, ami azt jelenti, hogy csak akkor renderelődik újra az adott kártya, amennyiben megváltoznak a props-ok.

- Ahogy a Gamelayout, ez a komponens is sok bemeneti adatot vár. Ezeket mindig megkapja, amikor használni szeretnénk.

```
function PlayerCard({
  player,
  chemImg,
  playerChemMap,
  displayedPosition,
  slotPos,
  isModal,
}) {
```

Kép 57 – PlayerCard props-ok

- A játékosok rarity-je alapján különböző dizájnúak a kártyák. Ehhez segít összekapcsolni a rarity-ket az importált kártyadizájnnal a rarityImgs.

```
const rarityImgs = {
  bronze: Bronze,
  silver: Silver,
  gold: Gold,
  icon: Icon,
  hero: Hero,
  toty: Toty,
  scream: Scream,
  flashback: Flashback,
};
```

Kép 58 - rarityImgs

- Mivel a kártyákon kapusoknál és mezőnyjátékosoknál különféle adatot jelenítünk meg, ezért renderelés előtt fontos, hogy ellenőrizzük, hogy kapus-e, vagy mezőnyjátékos az adott játékos.

- Illetve mivel minden rarity-nél más-más színűek a dizájn miatt a szövegek, ezért a játékos rarity-jét átadjuk a getRarityClass függvénynek, ami arra a célra szolgál, hogy rarity alapján színezi a szövegeket.

```
const isGK = player.player_positions === "GK";
const textClass = getRarityClass(player.rarity);
```

Kép 59 – kapus ellenőrzés és szövegek színezése

- Mivel számos játékos több pozícióban is tud játszani az alap pozíciója mellett, ezért a többi pozíciót is ki szeretnénk volna íratni a kártyára. Ehhez végig járjuk a játékos pozícióit, és hozzáadjuk a másodlagos pozíciók tömbjéhez, amennyiben épp nem abban a pozícióban játszik, mivel akkor az jelenik meg elsődleges pozíciónak.

```
const altPositions = [];  
player.player_positions.split(", ").forEach((element) => {  
  if (element !== displayedPosition(player, slotPos)) {  
    altPositions.push(element);  
  }  
});
```

Kép 60 – játékos másodlagos pozícióinak összegyűjtése

- Ezek után rendereljük a kártyát.

- Először is a kártya dizájnját rendereljük. Amennyiben van a játékosnak player_design_url-je, úgy az lesz a dizájn (ez csak utólag hozzáadott promonál van), ha pedig nincsen, úgy a rarityImgs segítségével a játékos rarity-je és az importált dizájnokat összekötve kapjuk meg az adott kártya dizájnját. Amennyiben ígyis sikerül error-ba futnunk, úgy sima arany kártyadizájnja lesz a játékosnak.

```
<img  
  loading="lazy"  
  src={player.card_design_url || rarityImgs[player.rarity]}  
  className={"cardDesign " + player.rarity}  
  alt="Card"  
  onError={(e) => {  
    e.currentTarget.onerror = null;  
    e.currentTarget.src = rarityImgs[player.rarity] || Gold;  
  }}  
>
```

Kép 61 – kártyadizájn renderelése

- Ezután a játékos alap adatainak, nemzetiségének, arcának stb. a renderelése történik.

```
<p className={`cardOverall ${textClass}`}>{player.overall}</p>
<p className={`cardPosition ${textClass}`}>
  {displayedPosition(player, slotPos)}
</p>
<p className={`cardAltPosition ${textClass}`}>
  {altPositions.map((pos, index) => (
    <span key={index}>{pos}</span>
  ))}
</p>

<img
  loading="lazy"
  className="cardImg"
  src={player.player_face_url}
  alt="Player Image"
  onError={(e) => {
    e.currentTarget.onerror = null;
    e.currentTarget.src = AltPlayerImg;
  }}
/>
<img
  loading="lazy"
  className="cardNationality"
  src={player.nation_url}
  alt="Nationality Image"
  onError={(e) => {
    e.currentTarget.onerror = null;
    e.currentTarget.src = NationAlt;
  }}
/>
<img
  loading="lazy"
  className="cardLeague"
  src={player.league_url}
  alt="League Image"
  onError={(e) => {
    e.currentTarget.onerror = null;
    e.currentTarget.src = ClubLeagueAlt;
  }}
/>

{player.rarity !== "icon" && player.rarity !== "hero" && (
  <img
    loading="lazy"
    className="cardClub"
    src={player.club_team_url}
    alt="Club Image"
    onError={(e) => {
      e.currentTarget.onerror = null;

      if (player.club_name === "Inter") {
        e.currentTarget.src = InterMilan;
      } else if (player.club_name === "AC Milan") {
        e.currentTarget.src = ACMilan;
      } else if (player.club_name === "Lazio") {
        e.currentTarget.src = Lazio;
      } else if (player.club_name === "Atalanta") {
        e.currentTarget.src = Atalanta;
      } else {
        e.currentTarget.src = ClubLeagueAlt;
      }
    }}
  </img>
)
}

<p className={`cardName ${textClass}`}>{player.short_name}</p>
```

Kép 62 – kártya alap adatainak renderelése

-A képeknél amennyiben nem sikerül a betöltés (ez amiatt van, mivel a használt adatbázisban sajnos nem mindenhez szerepel kép, amivel nem tudtunk mit kezdeni egy ilyen óriási játékos-adatbázisnál), abban az esetben mindig kezeljük az error-t. Minden esetben vagy egy placeholder kép jelenik meg, vagy vannak olyan csapatok, aminél tudtunk arról, hogy nem szerepelnek a képük az adatbázisban, és mivel elég nagy csapatokról van szó, ezért azokat magunknak importáltuk, és úgy használjuk.

- Amennyiben nem játékos kiválasztó modal-ban van éppen meghívva a PlayerCard, úgy a játékosok összhangja is szerepel a kártyákon. Ebben a chemImg és a playerChemMap külső függvények segítenek. Ezek segítségével egy, kettő, vagy három összhang csillag jelenik meg a kártyákon, a játékos összhangjától függően.

```
{!isModal && (  
  <img  
    loading="lazy"  
    src={chemImg(playerChemMap[player.player_id])}  
    className="chemStars"  
  />  
)}
```

Kép 63 – összhang csillagok renderelése

- Ezek után már csak a játékos statisztikái/statjai renderelésére van szükség. Itt a kapusoknak és a mezőnyjátékosoknak különféle stat-ok renderelődnek, attól függően, hogy az isGK true vagy false. Mivel ez egy rendkívül hosszú monoton folyamat, így csak egy adott statisztika renderelését mutatjuk be.

```
<div className="cardPlayerDribbling">  
  <p  
    className={`cardPlayerDribblingNumber ${getRarityClass(player.rarity)}`  
  >  
    {player.dribbling}  
  </p>  
  <p  
    className={`cardPlayerDribblingText ${getRarityClass(player.rarity)}`  
  >  
    DRI  
  </p>  
</div>
```

Kép 64 – egy stat renderelése a kártyán

- CSS segítségével pedig minden stat, adat pozícionálva van, hogy tökéletes és reszponzív legyen az UI. Itt egy példa erre:

```
.cardPlayerPassing {  
  position: absolute;  
  top: 59%;  
  right: 18%;  
  z-index: 3;  
  display: flex;  
  flex-direction: row;  
  width: 23%;  
  justify-content: space-between;  
  align-items: center;  
}
```

Kép 65 – stat pozícionálásához szükséges CSS formázás

2.4 A program tesztelése

- A program tesztelése során mind a backend-en szereplő végpontokat, mind a frontend-en szereplő főbb függvényeket teszteltük.

2.4.1 Backend teszt

- A végpontok tesztelésére a REST Client által nyújtott szolgáltatást vettük igénybe. Ide igyekeztünk a legtöbb végpontot behelyezni, és tesztelni működésüket. Ez lehetővé tette HTTP kérések futtatását a test.http fájlból, így gyorsan, és rendezetten ellenőrizhető volt a végpontok működése.

- A tesztelés célja az volt, hogy megbizonyosodjunk, hogy a végpontok helyesen működnek, helyes választ küldenek, és hogy működnek-e a hibakezelések.

-Íme néhány tesztet:

```
Send Request  
GET http://127.0.0.1:3000/api/ formations HTTP/1.1
```

Kép 66 - /api/formations HTTP request-je

- A /api/formations végpontra küldünk egy GET kérést.

- Elvárt működés: A szerver visszaadja az összes formációt.

```
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: http://127.0.0.1:3001
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 7335
ETag: W/"1ca7-Gzk6osy8jL8gTkz3GwFCZW4q/Dc"
Date: Tue, 28 Apr 2026 18:29:04 GMT
Connection: close

{
  "formations": [
    {
      "formation": "3-1-4-2",
      "layout": [
        {
          "pos": "GK",
          "x": 50,
          "y": 87
        },
        {
          "pos": "CB",
          "x": 25
        }
      ]
    }
  ]
}
```

Kép 67 - /api/formations HTTP request eredménye

- Teszt eredménye: HTTP 200 státusz, JSON tömb formációkkal, koordináták és pozíciók helyesen megjelennek.

```
Send Request
POST http://127.0.0.1:3000/api/users/login HTTP/1.1
content-type: application/json

{
  "username": "bobberito",
  "password": "Test1234"
}
```

Kép 68 - /api/users/login HTTP POST request-je

- A /api/users/login végpontra küldünk egy POST kérést, aminek body-jába beírjuk a felhasználónevet, és a jelszót.

- Elvárt működés: amennyiben a felhasználónév és a jelszó is helyes, úgy egy 200-as státuszú választ, valamint a session létrejöttét, és egy Logged in

üzenetet, amennyiben valamelyik helytelen, abban az esetben pedig egy hibaüzenetet arról, amelyik rossz, és egy 400-as státuskódú választ helytelen felhasználónév esetén, és 403-as státuskódú választ helytelen jelszó esetén.

- Amennyiben jó felhasználónevet és jelszót adtam meg:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: http://127.0.0.1:3001
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 23
ETag: W/"17-EIjjQneeu5F71qsy6ljtAOYomk"
Set-Cookie: connect.sid=s%3A1AnJIGnDIgaVnck91AYGHZglwV
X9MHqkb.Tn1RRxJddkISXXRKOMitV%2FZnq0IRGEN07vO5SIQH0A
Q; Path=/; HttpOnly; SameSite=Lax
Date: Tue, 28 Apr 2026 18:46:26 GMT
Connection: close

{
  "message": "Logged in"
}
```

Kép 69 - /api/users/login POST request első eredménye

- Teszt eredménye: 200-as státuskódú válasz, session létrejön, Logged in üzenet.

- Amennyiben jó felhasználónevet, de rossz jelszót adtam meg:

```
HTTP/1.1 403 Forbidden
X-Powered-By: Express
Access-Control-Allow-Origin: http://127.0.0.1:3001
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 28
ETag: W/"1c-1DgWgAxsia6W2YD8mP0Sbp4yZkI"
Date: Tue, 28 Apr 2026 18:43:07 GMT
Connection: close

{
  "message": "Wrong password"
}
```

Kép 70 - /api/users/login POST request második eredménye

- Teszt eredménye: 403-as státuskódú válasz, session nem jön létre, Wrong password üzenet.

- Amennyiben rossz felhasználónevet, és jó/rossz jelszót adtam meg (itt nem számít, mivel amennyiben rossz a felhasználónév, nem megy tovább a jelszó vizsgálatára):

```
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Access-Control-Allow-Origin: http://127.0.0.1:3001
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 30
ETag: W/"1e-aUr1HPHIQJnh0bG+bz7wDq6q0ZE"
Date: Tue, 28 Apr 2026 18:45:37 GMT
Connection: close

{
  "message": "Invalid username"
}
```

Kép 71 - /api/users/login POST request harmadik eredménye

- Teszt eredménye: 400-as státuszkódú válasz, session nem jön létre, Invalid username üzenet.

2.4.2 Frontend teszt

- A programban szereplő főbb, jól tesztelhető függvények ellenőrzésére QUnit alapú unit tesztek kerültek implementálásra.

- A tesztek célja, hogy biztosítsák a játékmenethez kapcsolódó számítások megbízható működését.

- A tesztek böngészőben futnak, a QUnit 2.20.0 framework-ön.

-A tesztelt modul/fájl a utilities.testable.js. Ez a fájl a kód tesztelhető verziója, amire azért volt szükség, mivel az alap utilities.js-ben szerepeltek olyan függvények is, amiben képek is szerepelnek, és azon nem tudtuk futtatni a tesztet.

- A tesztek egyetlen modulban vannak, összesen 16 teszteset, és 43 assertion.

Íme pár példa belőle:

```
QUnit.module("Játékban szereplő függvények tesztelése");

// ChemImg
QUnit.test("Chemistry csillag megjelenik", (assert) => {
    assert.ok(chemImg(3) !== undefined);
    assert.ok(chemImg(2) !== undefined);
    assert.ok(chemImg(1) !== undefined);
    assert.ok(chemImg(0) !== undefined);
});

// RatingStars
QUnit.test("Rating csillagok működnek", (assert) => {
    assert.strictEqual(ratingStars(85), "★★★★★");
    assert.strictEqual(ratingStars(80), "★★★★☆");
    assert.strictEqual(ratingStars(70), "★★★★☆");
    assert.strictEqual(ratingStars(66), "★★★★☆");
    assert.strictEqual(ratingStars(10), "★★★★☆");
    assert.strictEqual(ratingStars(0), "★★★★☆");
});

// GraphProgress
QUnit.test("Grafikon működik", (assert) => {
    assert.strictEqual(calculateGraphProgress(0, 0), 0);
    assert.ok(calculateGraphProgress(50, 50) <= 360);
});

// DisplayedPosition
QUnit.test(
    "Jó pozíció jelenik meg a játékos csapatba helyezésekor",
    (assert) => {
        const player = { player_positions: "ST, LW" };

        assert.strictEqual(displayedPosition(player, null), "ST");
        assert.strictEqual(displayedPosition(player, "LW"), "LW");
        assert.strictEqual(displayedPosition(player, "GK"), "ST");
    },
);
```

Kép 72 – betekintés a unit tesztbe

- A tesztelés eredménye: minden teszt sikeresen lefutott, nem történtek hibák.

QUnit Tesztek		
☐ Hide passed tests ☐ Check for Globals ☐ No try-catch		
Filter:		Go Module: All modules
QUnit 2.20.0; Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36		
16 tests completed in 15 milliseconds, with 0 failed, 0 skipped, and 0 todo. 43 assertions of 43 passed, 0 failed.		
1. Játékban szereplő függvények tesztelése: Chemistry csillag megjelenik (4)	Rerun	2 ms
2. Játékban szereplő függvények tesztelése: Rating csillagok működnek (6)	Rerun	0 ms
3. Játékban szereplő függvények tesztelése: Grafikon működik (2)	Rerun	0 ms
4. Játékban szereplő függvények tesztelése: Jó pozíció jelenik meg a játékos csapatba helyezésekor (3)	Rerun	0 ms
5. Játékban szereplő függvények tesztelése: Reward értékek működnek (5)	Rerun	1 ms
6. Játékban szereplő függvények tesztelése: Draft kitöltésének ellenőrzése (2)	Rerun	0 ms
7. Játékban szereplő függvények tesztelése: Klub értékének kiszámolása (1)	Rerun	0 ms
8. Játékban szereplő függvények tesztelése: Objective teljesítésének ellenőrzése (1)	Rerun	0 ms
9. Játékban szereplő függvények tesztelése: Objective progress százalék működik (3)	Rerun	0 ms
10. Játékban szereplő függvények tesztelése: Kártya szövegeinek színe működik (3)	Rerun	0 ms
11. Játékban szereplő függvények tesztelése: SBC feltételek (1)	Rerun	0 ms
12. Játékban szereplő függvények tesztelése: SBC feltételek ketté szedése - min/max és value (2)	Rerun	0 ms
13. Játékban szereplő függvények tesztelése: SBC feltételek teljesítésének ellenőrzése (3)	Rerun	0 ms
14. Játékban szereplő függvények tesztelése: SBC aktuális csapat állása (4)	Rerun	1 ms
15. Játékban szereplő függvények tesztelése: Pack játékosainak értéke (1)	Rerun	0 ms
16. Játékban szereplő függvények tesztelése: Pack-ben csak duplicate van-e (2)	Rerun	0 ms

Kép 73 – QUnit teszteredmények

2.5 Továbbfejlesztési lehetőségek

- A program fejlesztése során ügyeltünk arra, hogy minél könnyebben lehessen a jövőben új promo-kat hozzáadni a játékhoz, illetve új paklikat stb.
- Már jelenleg is az admin felületen lehetőség van arra, hogy új promo lapokat hozzáadjunk a játékhoz, saját dizájnnal, és stat-okkal is akár. Ezen az alapon könnyen tovább lehet menni akár a jövőben, hogy új SBC-ket, paklikat, vagy bármit igazából hozzá lehessen adni a jelenlegi játékhoz.
- Ezen kívül számos lehetőséget kínál még szerencsére a játék továbbfejlesztése:

- VS. játékmód: felhasználók csapatainak “meccse”
- Játékos piac: lehessen egymás között eladni, és vásárolni játékosokat

- Draft vs. Bot: Lehesen a draft csapatunkat összemérni egy random generált csapattal

3. Felhasználói dokumentáció

3.1 A program célja

- A fő cél, hogy A felhasználó minél erősebb kártyákat gyűjtsön össze a csapatában a lehető legtöbb összhanggal. Ennek a célnak az elérésére több mód is van a játékon belül, az a felhasználón múlik, hogy melyiket/melyikeket használja, hogy közelebb jusson a célhoz.

3.2 Rendszer követelmények

3.2.1 Processzor (CPU)

- Minimum követelmény: Bármilyen modern kétmagos processzor (Intel Core i3 10. Generáció és felette vagy Amd Ryzen 3)
- Ajánlott követelmény: Bármilyen négymagos processzor (Intel Core i5 vagy Amd Ryzen 5)

3.2.2 Videókártya (GPU)

- Minimum követelmény: Bármilyen hardver gyorsítást támogató beépített videokártya (Intel UHD vagy Amd Radeon Graphics)
- Ajánlott követelmény: Bármilyen modern különálló videokártya (NVIDIA GTX 900-as széria vagy AMD Radeon R9 300-as széria)

3.2.3 Memória (RAM)

- Minimum követelmény: 4gb ddr3-as memória
- Ajánlott követelmény: 8gb ddr4-es memória

3.2.4 Hálózat és Böngésző

- Hálózati követelmény: Stabil internet kapcsolat
- Böngésző minimum követelmény: Chrome 110-es verzió és felette vagy Safari 16-os verzió és felette
- Böngésző ajánlott követelmény: A legfrisebb chromium alapú böngésző

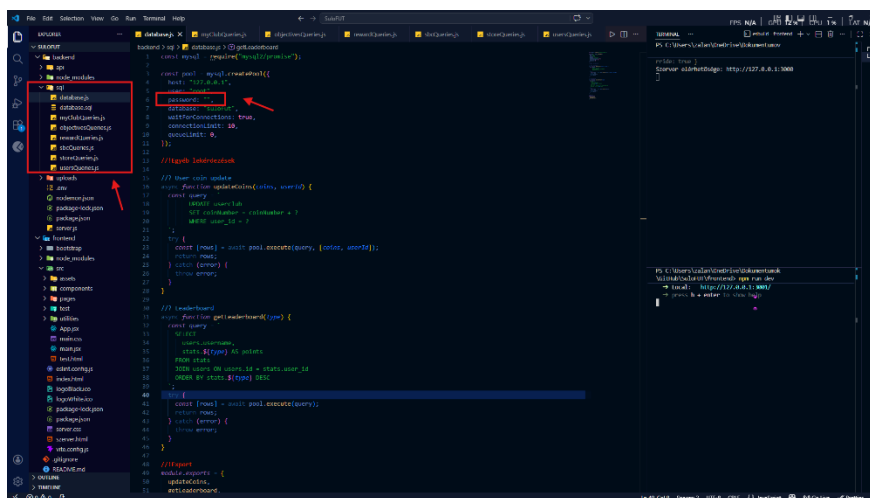
3.3 Telepítés

- Első lépés a XAMPP Control Panel telepítése, amit a következő oldalon tud megtenni:



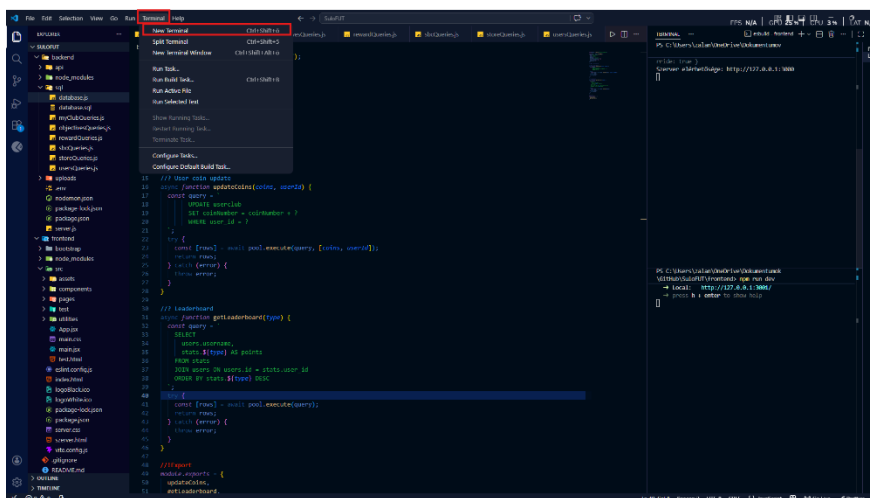
Kép 74 - XAMPP telepítése

- Ha telepítette, a következő lépés, hogy meg kell nyitni az alkalmazást tartalmazó mappát egy általunk választott forráskód szerkesztőben (Ebben az esetben Visual Studio Code). Ha használ jelszót a phpMyAdmin felületéhez akkor azt a következő helyen tudja megadni:



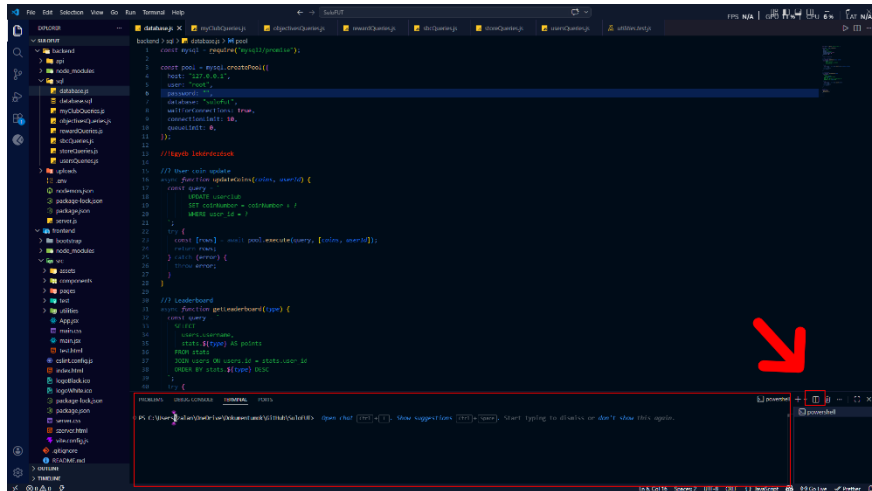
Kép 75 - phpMyAdmin jelszó megadása

- Ezután nyisson meg egy terminált a szerver elindításához



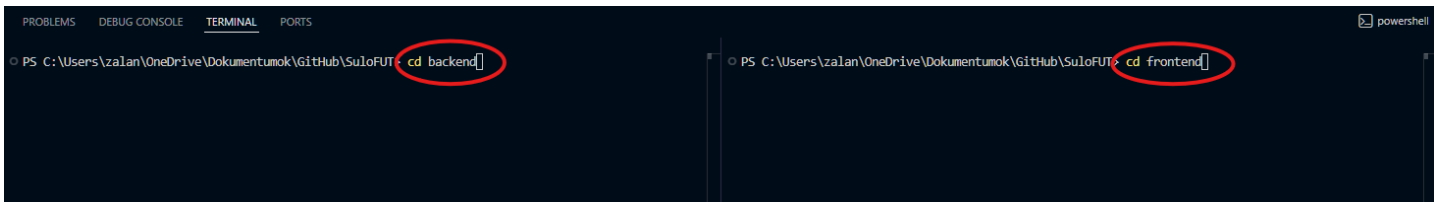
Kép 76 - terminál megnyitása

- Látni fog az oldal alján egy terminál fület, ott a “Split terminal” opciót kell használni (A nyíl erre mutat)



Kép 77 - split terminál

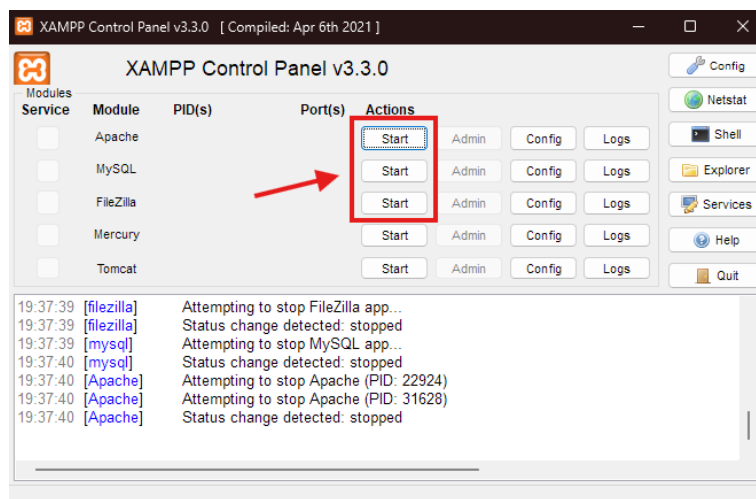
- A két különböző terminálban a következő két sort kell beírni elsőnek:



Kép 78 - terminálon belül mappákba belépés

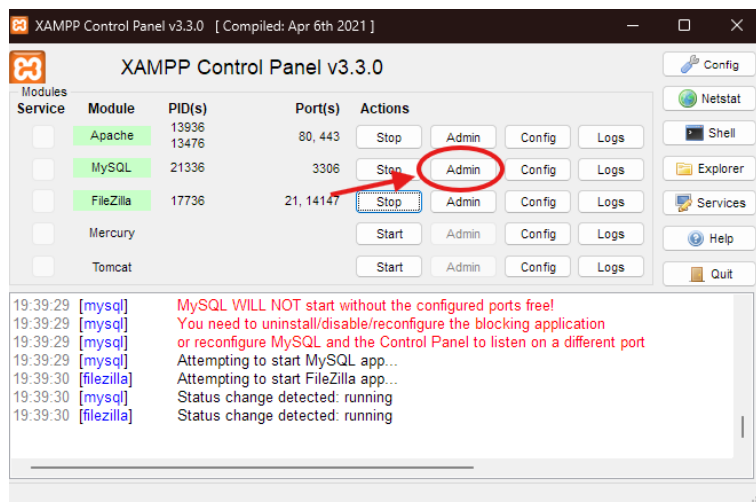
- Innentől mindkét terminálban ugyanaz a feladat. Elsőnek az “npm i” vagy “npm install” parancsot kell beírni, megvárja a felhasználó a telepítést, ha ez kész van akkor az “npm run dev” parancssal tudja elindítani a szerveret

- Ezután nyissa meg a XAMPP Control Panel-t ahol a következő dolgokat kell elindítani:



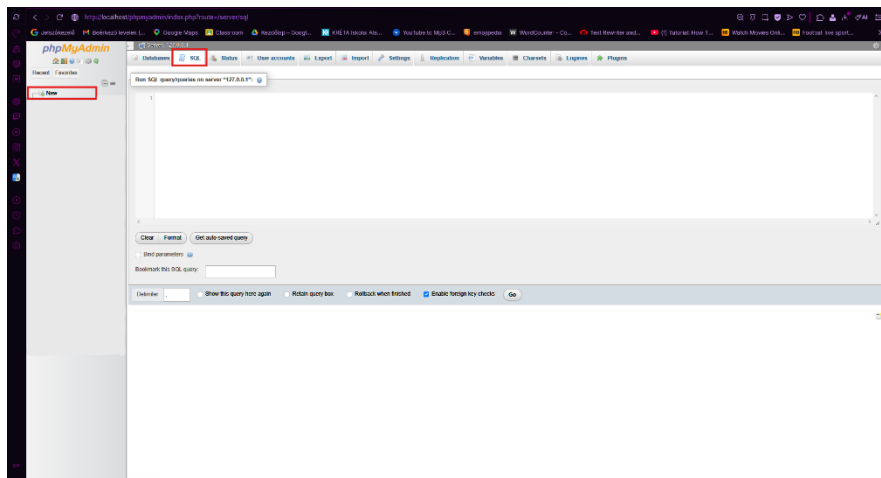
Kép 80 - elindítandó service-ek

- A következő lépésnél a phpMyAdmin felületre lesz szükségünk, amit a legkönnyebben így lehet elérni:



Kép 81 - phpMyAdmin megnyitása

- A phpMyAdmin felületen rá kell nyomni a “New” gombra aztán pedig az “SQL” gombra kattintva be tudjuk illeszteni az adatbázisunkat



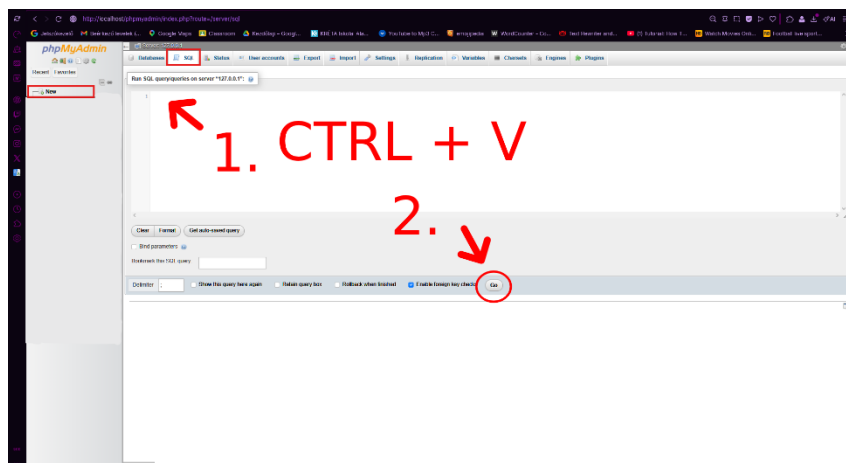
Kép 82 - adatbázis beillesztés 1. lépés

- Másoljuk ki az adatbázis a forráskódban található a database.sql fájlból:



Kép 83 - adatbázis beillesztés 2. lépés

- Illesszük be a phpMyAdmin oldalára majd nyomjunk rá, hogy “Go” vagy “Indítás”:



Kép 84 - adatbázis beillesztés 3. lépés

- Utolsó lépés a Node telepítése, amit a következő oldalon tud megtenni: <https://nodejs.org/en/download>

3.4 Használat

- Az oldal használatához a felhasználónak egy böngészőre lesz szüksége, a keresősávban a következő link beírásával juthat el az oldalra:

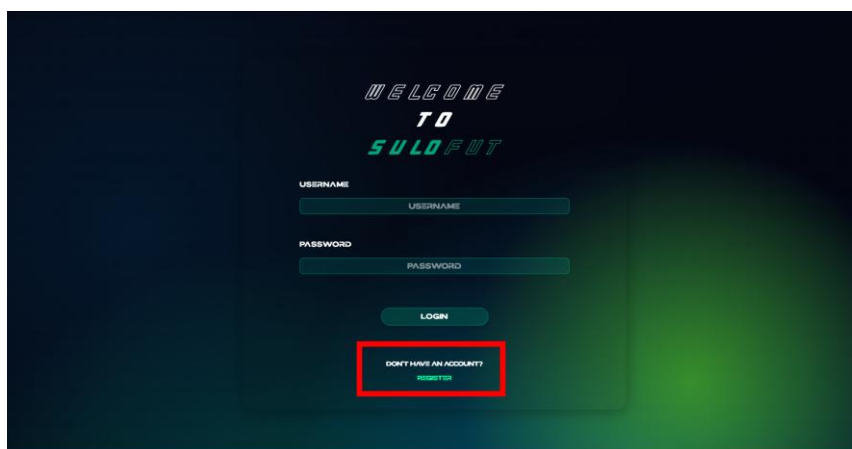
<http://127.0.0.1:3001/>

- Ezt követően a felhasználó regisztrálni is tud (Aminek a módját a Fő funkciók ismertetésekor részletesen elmagyarázzuk)

3.5 Fő funkciók

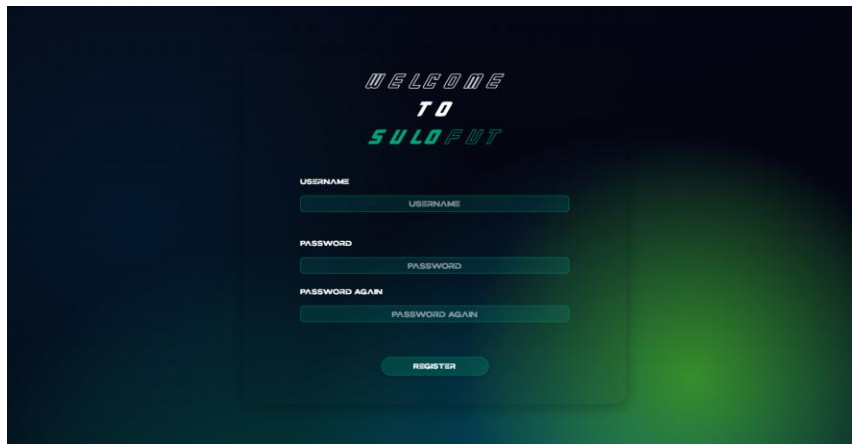
3.5.1 Saját felhasználó

- Saját felhasználó készítése, ahova az összes addigi előrehaladás elmentésre kerül. Az alábbi képen a piros kockában lévő linkre kattintva teheti meg a felhasználó, hogy saját fiókot regisztrál:



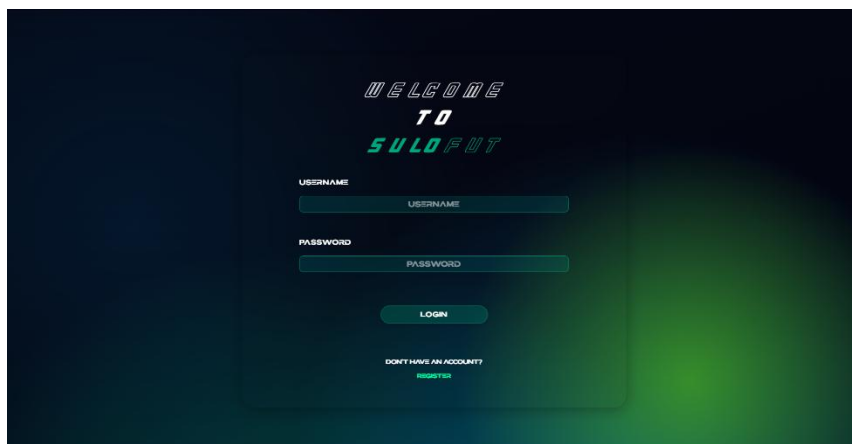
Kép 85 - regisztrációs felület elérése

- Ezt követően a regisztrációs felületen fogja találni magát, ahol egy felhasználónév és a jelszó kétszeres megadása után regisztrált felhasználónak minősül



Kép 86 - regisztrációs felület

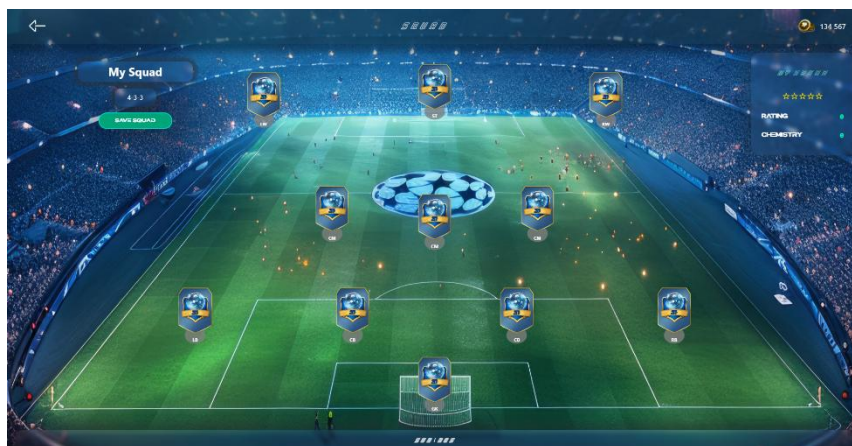
- Következő lépés a bejelentkezés, ahol az előbb említett regisztrációban használt felhasználó név és jelszó kombinációval be tud lépni a játékba.



Kép 87 - bejelentkezési felület

3.5.2 Squad

- Minden felhasználónak van egy saját csapata, amit tetszése szerint tölthet fel a megszerzett játékosaiival, ez a squad menüpontban érhető el. Adhat tetszés szerint egy nevet is a csapatának, illetve meg tudja változtatni a formációt is. Illetve van cserepad és tartalékok is, a cserepad a csapat összértékelésén is tud segíteni bár kisebb mértékben, mint a kezdőcsapat. FONTOS: A csapatjátékosai a “save squad” gomb megnyomásával mentődnek el a használt csapatban.



Kép 88 - squad képernyő

- A jobb oldalon látható egy rating és egy chemistry statisztika. A rating a játékosok átlag értékeléséből kerül kiszámolásra (mindkettő fontosabb lesz későbbiekben az SBC és a Draft játékmódoknál). A chemistrybe csak akkor számít bele egy játékos, ha egy olyan pozíción van elhelyezve, amiben tud játszani (Ez az értékelése alatt található). Ha ez stimmel következő táblázat alapján van kiszámolva az összhang:

NATION

2 = 

5 = 

8 = 

LEAGUE

3 = 

5 = 

8 = 

CLUB

2 = 

4 = 

7 = 

ICON

2 = 

4 = 

6 = 

HERO

Ha egyezik a liga:

 /Hero

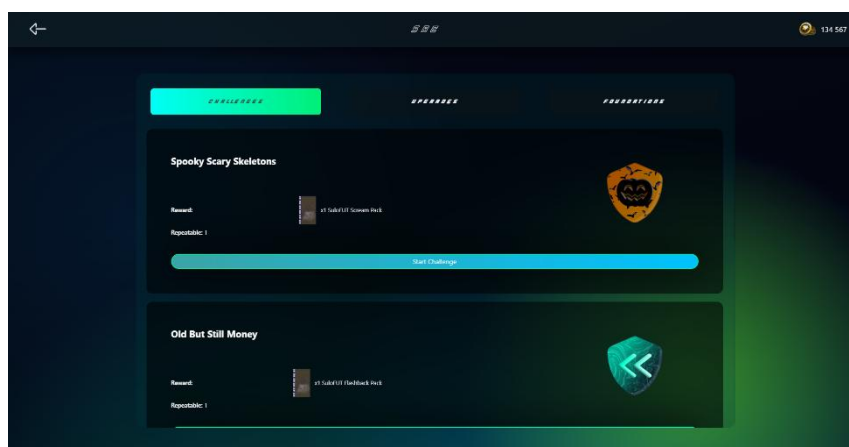
*Minden kon ritkaságú játékos alapból 3 chemistry ponton van

*Minden Hero ritkaságú játékos alapból 3 chemistry ponton van

Kép 89 - chemistry magyarázat grafika

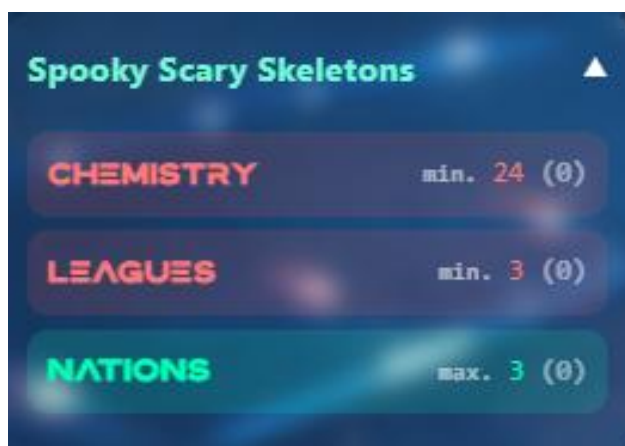
3.5.3 Squad Building Challenge

- Az SBC menüpontban találhatóak a Squad Building Challenge-k, ezek különböző kihívások, amiket egy csapat építésével kell megoldani a saját klubbodból, gondoljunk úgy rá, mint egy puzzle csak focistákkal. Ezekért a felhasználó paklikat kaphat.



Kép 90 - sbc képernyő

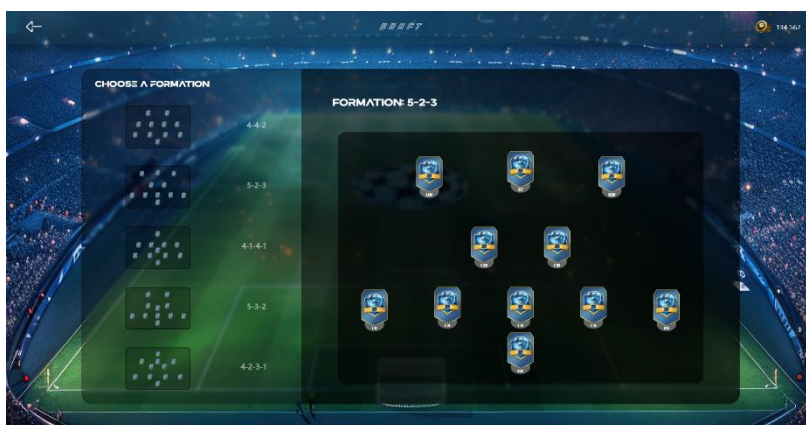
- A “Start Challenge” gombra kattintva bedob minket egy oldalra, ami nagyon hasonló, mint a squad menüpontban, két kivétellel: Nincs cserepad, és a bal oldalt van egy követelmény ablak, ezt megtekintve láthatjuk a feltételeket, aminek a csapat meg kell feleljen annak érdekében, hogy teljesítsük a kihívást. Zölddel jelenik meg a feltétel, aminek megfelel a csapat és pirossal, aminek nem.



Kép 91 - feltételek menüpont

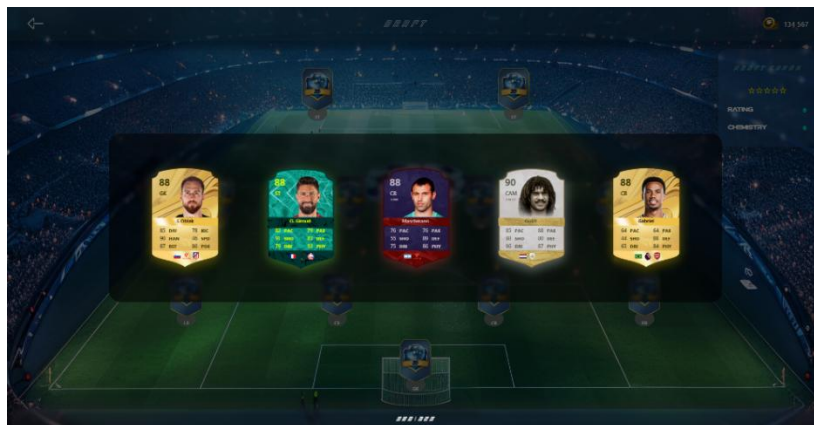
3.5.4 Draft

- A Draft menüpontban található az egyik legizgalmasabb játékmód, amiben 5 véletlenül generált játékosok közül kell választani minden egyes pozícióra, és így kell a lehető legjobb csapatot összerakni. A felhasználó ez alapján jutalmakat kap, ami lehet coin illetve pakli is. Első lépésben a felhasználónak kell választani egy formációt az 5 véletlenszerűen kisorsolt formációból



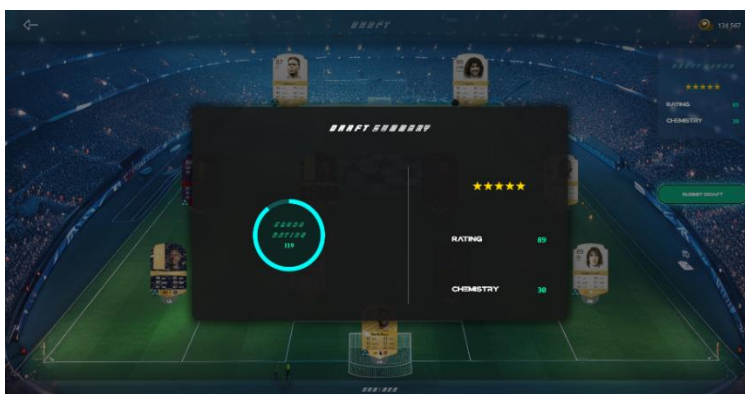
Kép 92 - draft formáció választó képernyő

- Ezután egy csapatkapitányt kell választania, ez pozíciótól függetlenül 5 véletlenszerű játékos, ha nem található olyan poszt a formációban, amiben a játékos tud játszani akkor automatikusan a cserepadra kerül

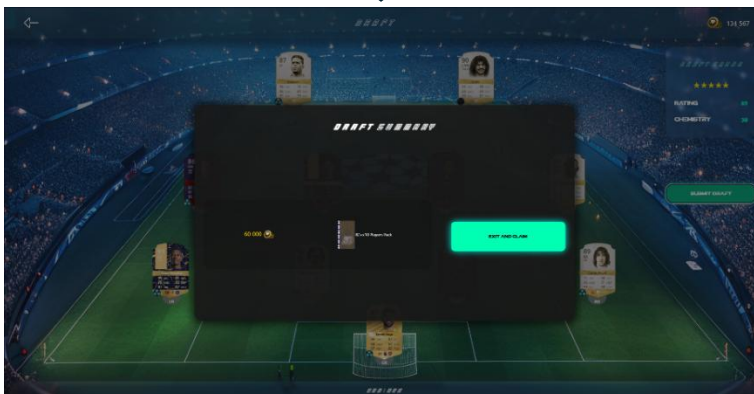


Kép 93 - kapitány választás

- Ezt követően minden pozíciónál ezt végre kell hajtani, ügyelve arra, hogy a rating mellet a chemistry is a legideálisabb legyen, mivel a felhasználó ezek alapján kapni fog egy összesített értékelést a draft-jára, a jutalmak is ez alapján vannak kiszámolva.



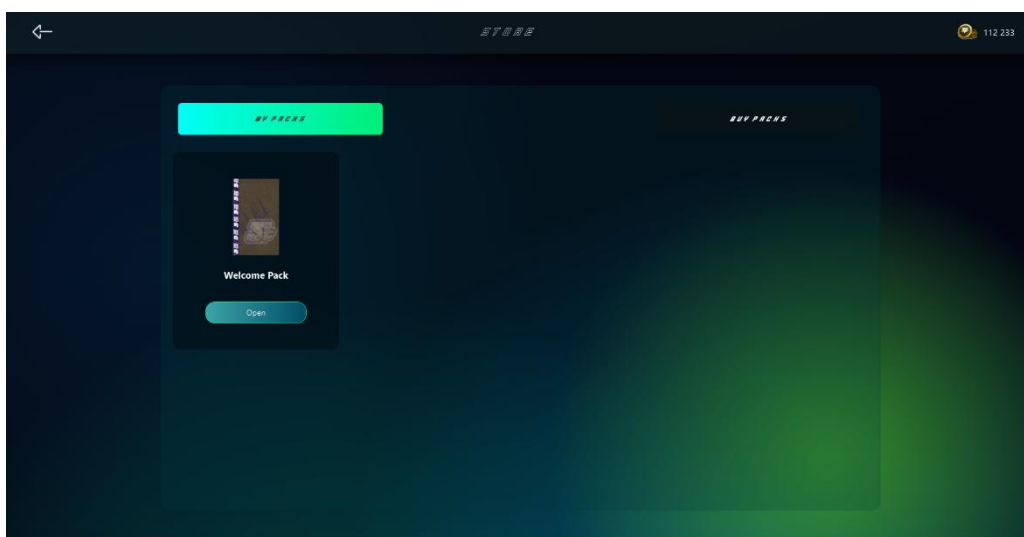
Kép 94 - draft kiértékelés



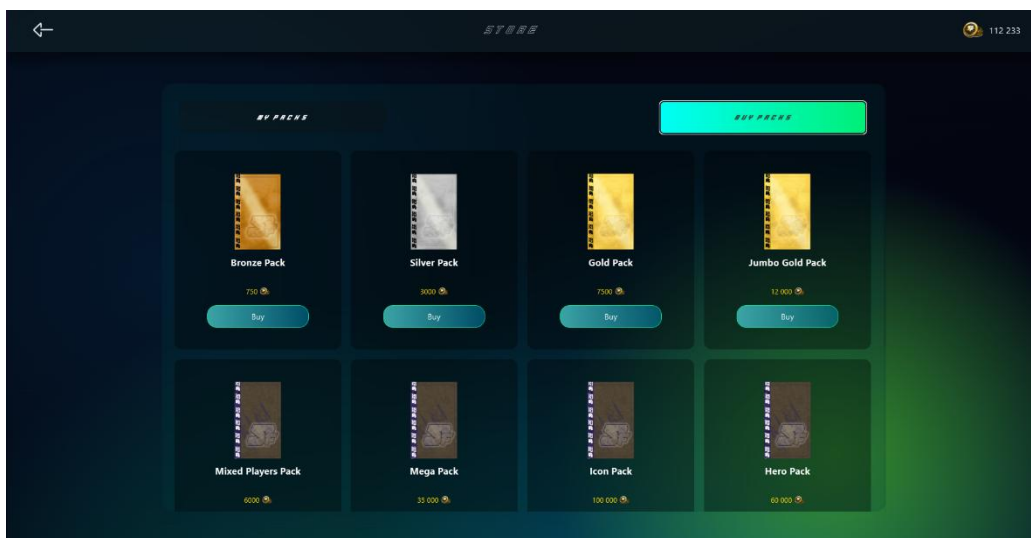
Kép 95 - draft jutalmak

3.5.5 Store

- Itt találhatóak a megszerzett paklik és az éppen vásárlásra elérhető paklik. A vásárlás a játék beépített “coin” funkciójából történik, amit a játékos a játékmódok használatával és az azokból szerzett paklikban található (vagy a klubban eltárolt) játékosok eladásával tud a felhasználó növelni. A játék kezdésekor mindenkinek elérhető egy “welcome pack”, ami segít a játékban elindulni.

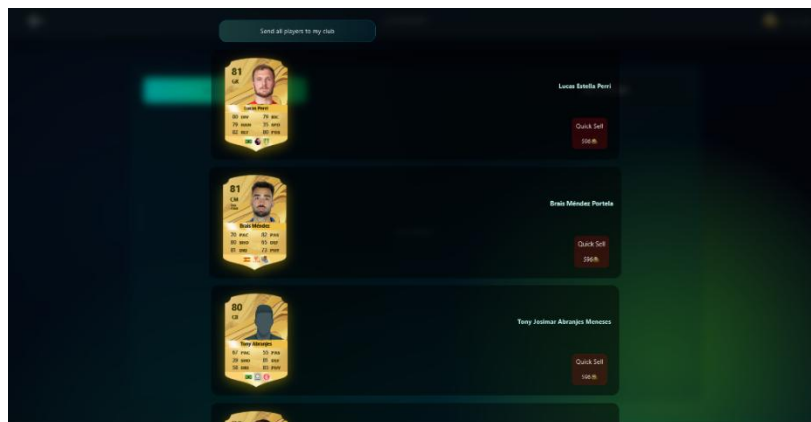


Kép 96 - my packs képernyő

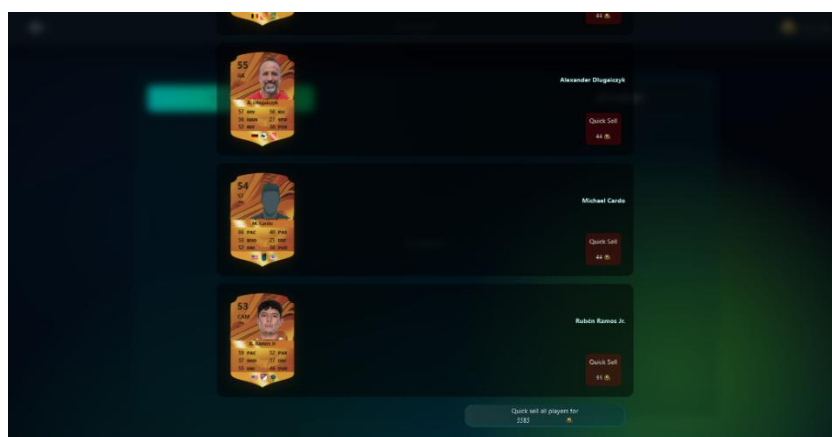


Kép 97 - vásárolható paccek

- A pakli kinyitásával megjelennek a nyitott játékosok értékelés szerint csökkenő sorrendben. Ekkor 3 opciója van a játékosnak. “send all players to my club” – ez eltárolja az összes kinyitott játékost a felhasználó klubbjában. “quick sell” – itt egyesével lehet eladni a játékosokat, akik nem kellenek, vagy az oldal alján a “quick sell all players” opcióval az összeset el tudja adni.

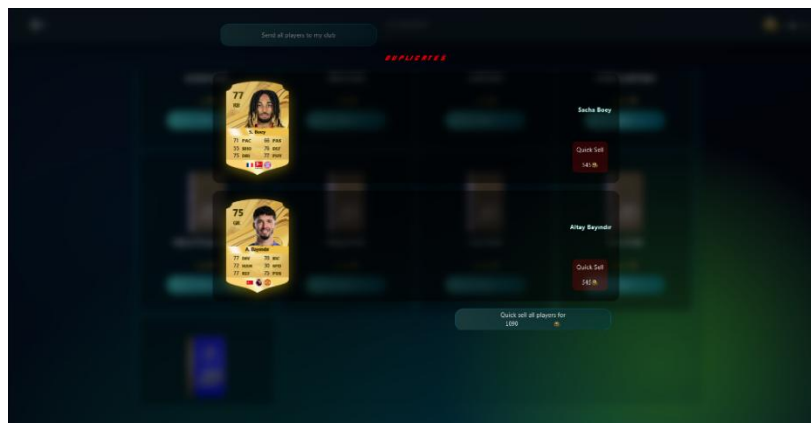


Kép 98 - pack nyitó képernyő



Kép 99 - quick sell all opció

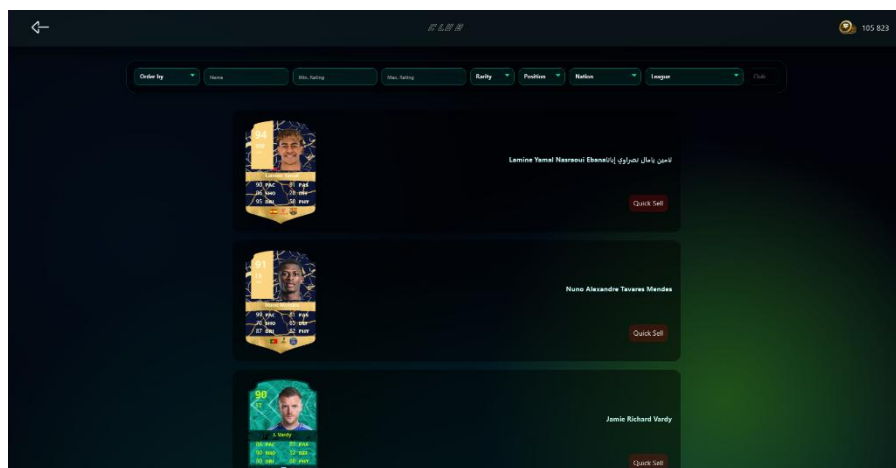
- A játék automatikusan ellenőrzi, hogy egy játékos meg van-e és ha igen akkor azt külön jelzi, ilyen játékosok esetében csak az eladás lehetséges



Kép 100 - duplicate játékosok

3.5.6 Club

- A club menüpont tartalmazza az összes megszerzett játékost, itt el lehet adni azokat, akikre már nincs szükség vagy coin-t szeretnénk belőle szerezni. Ez alaptól értékelés szerint van elrendezve, de a filter funkcióval kedvünk szerint tudjuk ezt változtatni.



Kép 101 - my club képernyő

3.5.7 Filter

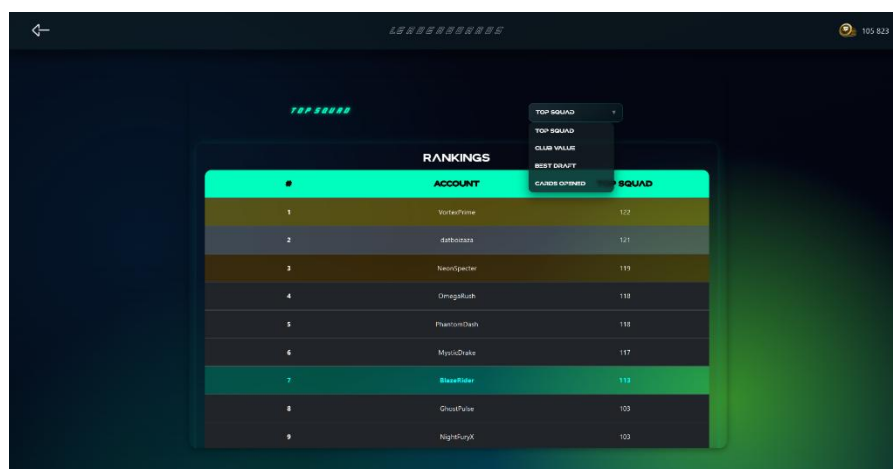
- Egy filter rendszer megkönnyebbíti a specifikus játékosok keresését. 9 opciónk van ami szerint tudunk szűrni, ezek a következők: Sort by – ez elrendezi a játékosokat értékelés vagy érték szerint, Name – név alapján rá lehet keresni a játékosra, Min és Max Rating – minimum és maximum értékelést tudunk megadni a keresett játékosainknak, Rarity – a játékos ritkaságára utal pl.: Gold, Position – az alapján keres hogy a megadott pozíción képes-e játszani az adott játékos, Nation – a játékos nemzetisége alapján történő keresés, League – a játékos ligája alapján történő keresés, Club – a játékos klubja alapján történő keresés.



Kép 102 - filter funkció

3.5.8 Leaderboard

- A leaderboard képernyő segítségével versenyezhetnek vagy összehasonlíthatják barátaikkal a felhasználók a saját előrehaladásukat különböző kategóriákban amik a következők: Top squad – Ez a kategória a felhasználók squad menüpontban épített csapataikat a rating és a chemistry összeadásával, Club value – Itt a felhasználók összes jelenleg birtokolt játékosainak értékét mérhetik össze, Best draft – A játékos legnagyobb elért értékelése a “Draft” játékmódban, Cards opened – Ez az összes kinyitott kártya számára vonatkozik a fiók létrehozása óta.

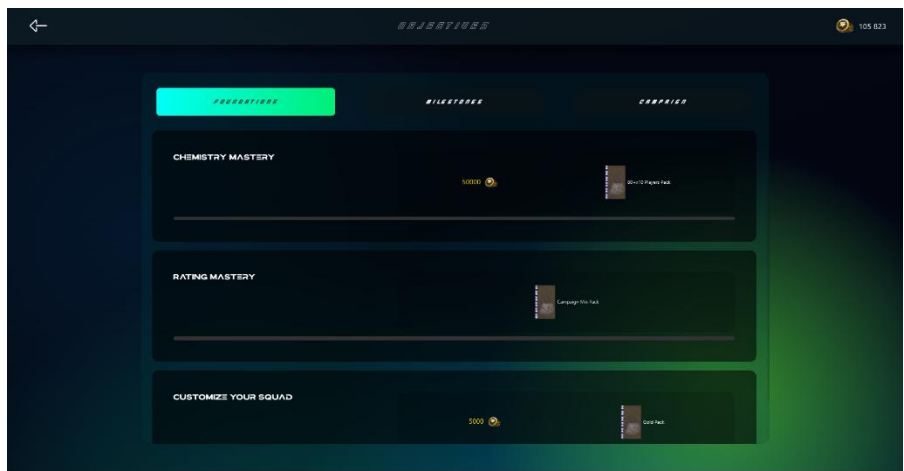


#	ACCOUNT	SQUAD
1	VortexPrime	122
2	darbassara	121
3	NeoSpecter	119
4	OmegaRuth	118
5	PhantomDash	118
6	MyxioDrake	117
7	BaseRider	113
8	GhostPulse	103
9	NightSuryX	103

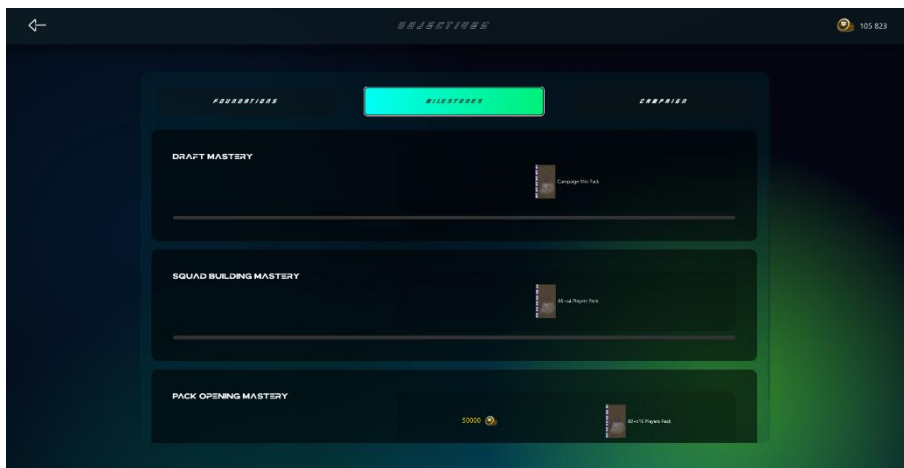
Kép 103 - leaderboard képernyő

3.5.9 Objectives

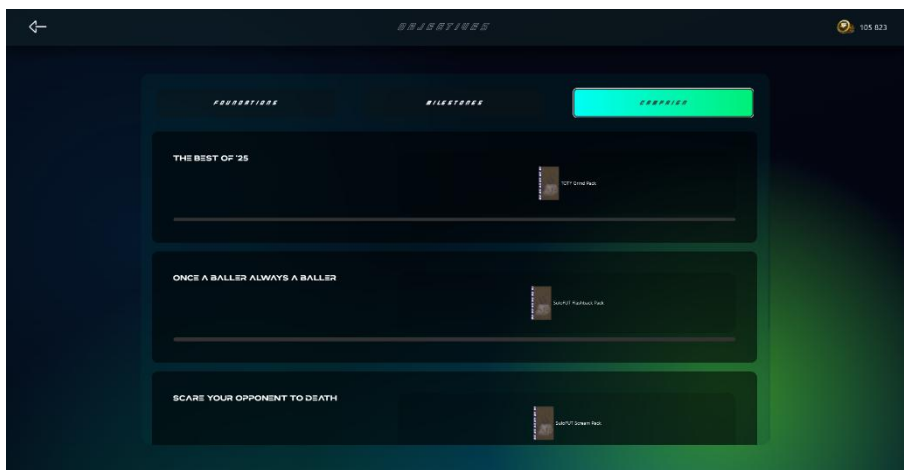
- Az objectivek különböző feladatok, amit a felhasználó a különböző játékmódok játszásával tud teljesíteni, ebből 3 kategória van. A foundations megismerteti a játékkal a játék alapvető mechanikáit és bemutatja őket. Ezek a jutalmak adnak egy kezdő löketet a felhasználónak. A milestones ösztönöz a játékra hiszen minél többet játszol vele annál közelebb jutsz ezek teljesítéséhez. A seasonal objectivek a játékban lévő promóciókkal kapcsolatosak így, ha valamelyik szimpatikus megéri őket megcsinálni.



Kép 105 - foundations



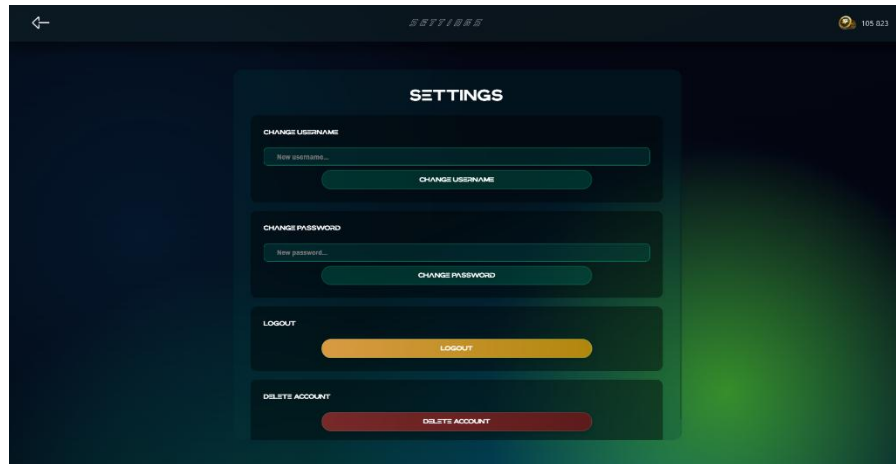
Kép 104 - milestones



Kép 106 - seasonal

3.5.10 Settings

- Itt a játékos meg tudja változtatni a felhasználónevét, illetve a jelszavát is. Ezen felül ki tud jelentkezni, vagy tudja véglegesen törölni a fiókját, ha nem nyerte el a tetszését a játékunk.



Kép 107 - settings képernyő

4. Csoportmunka megvalósítása

4.1 A kód

- Nem osztottuk fel konkrétan frontend és backend részekre, segítettük egymást ahogy tudtuk. Nem ragaszkodott senki ahhoz, hogy csak frontend, vagy csak backend pozíciót vegyen fel.
- Bugja Botond: Frontend, Backend, Adatbázis.
- Kakas Zsolt: A bonyolultabb backend végpontok megvalósítása, Adatbázis.

4.2 Egyebek

- Az alkalmazás rengeteg más egyéb elemet is igényelt a kódoláson túl, gondolván itt a grafikus elemekre, játékos adatbázisra és még sok másra.
- Bugja Botond: Játékos adatbázis struktúrájának elkészítése, illetve promóciós lapok hozzáadása
- Kakas Zsolt: A grafikus elemek elkészítése, mint pl.: Pack és kártya design, SBC ikon design és még sok más, illetve a játékos adatbázis feltöltése Icon és Hero játékosokkal

4.3 Menete

- A fontosabb döntéseket általában közösen, átbeszélve hoztuk meg. Legyen szó adatbázisról vagy bármilyen funkcióról. Segítettük egymást, ahol tudtuk, és folyamatos kommunikációban voltunk, ami megkönnyítette a projektmunka folyamatát.

5. Összegzés

5.1 A kitűzött célok teljesítése

- A szakmai célkitűzésünk az volt, hogy egy új programozási nyelvet tanuljunk meg a React formájában. Összeségében úgy gondoljuk, hogy ezt a célt teljesítettük, ami a játék teljes funkciókészletét illeti, egy kicsit hosszabb időkerettel talán még több funkciót be tudtunk volna iktatni, mert az ötletek meg voltak rá, de összeségében így is az alap elvárásunkat teljesítettük.
- Illetve a másik nagy célunk az volt, hogy gördülékenyen működjön a csoportmunka, ezt is sikerült teljesíteni. Mindenki végezte a dolgát, közösen építettük fel a projekt elkészítésének menetét, és közösen egymásra építve is valósítottuk meg.

5.2 Nehézségek a megvalósításban

- Számtalan nehézséggel néztünk szembe konkrétan az első naptól kezdve, mivel még abban sem voltunk biztosak, hogy használhatjuk valós játékosok adatait, szerencsére engedélyezve lett, viszont, ha ez nem lett volna, valószínűleg fel is adhattuk volna az egészet.
- A reszponzivitás sem volt egy egyszerű feladat, mivel nagyon sok grafikus elemmel dolgoztunk, ami viszonylag limitált abban, hogy mennyire tud átalakulni mielőtt használhatatlan lesz, vagy egyszerűen nem esztétikus. Illetve ezt a sok grafikus elemet egyesével meg kellett dizájnolni, lehet itt szó pack dizájnokról, kártya dizájnokról, a menüben található mini ikonokról, vagy még sok másról.

- Megemlítenénk még a players.json fájlt, ami az alapja az egész alkalmazásnak, mivel ez tartalmazza az összes játékos adatát. Ennek felépítése nehezebb feladat volt, mint azt alaphól gondoltuk, a létező erőforrások hiányosak voltak ezzel kapcsolatban, szóval nagyon sok adatot manuálisan, illetve scriptekkel kellett beilleszteni az adatbázisba.

6. Ábrajegyzék

Kép 1 - Sql relációk	7
Kép 2 - Játékos JSON objektum	8
Kép 3 - Formáció JSON objektum.....	9
Kép 4 - A program folyamatábrája.....	11
Kép 5 - A backend mappa.....	12
Kép 6 - A frontend mappa.....	14
Kép 7 - játékosszám feltételek.....	15
Kép 8 - összes játékosszám feltétel	16
Kép 9 - inPosition check	16
Kép 10 - calculateChemistry function 1.....	17
Kép 11 - chemistry grafikon	18
Kép 12 - active players for Each	18
Kép 13 - countok kiszámolása	19
Kép 14 - player.map 1.....	19
Kép 15 - players.map 2	20
Kép 16 - players.map 3	20
Kép 17 - response	21
Kép 18 - a játékosok elmentése egy tömbbe	22
Kép 19 - kezdő 11 hosszának ellenőrzése	22
Kép 20 - rating-ek "kiszédése"	23
Kép 21 - rating megkapása számolások után.....	23
Kép 22 - rating response	23
Kép 23 - pos és data a random játékos generálásnál	24
Kép 24 - kiválasztott játékosok id-ja a draft-ban	24
Kép 25 - pozíció csoportok	24
Kép 26 - getPositions függvény	25
Kép 27 - rating range-ek a draft-ban	26
Kép 28 - 5 random játékos generálása	27
Kép 29 - packWeight tábla.....	28
Kép 30 - pack tábla.....	28
Kép 31 - lekérdezések	28
Kép 32 - get metódus inicializálása.....	29
Kép 33 - randompack function inicializálása	30
Kép 34 - rarityCount	31
Kép 35 - rarityCount ellenőrzése.....	32
Kép 36 - selectedRarity	33
Kép 37 - validPlayers	33
Kép 38 - kiválasztott játékosok pusholása.....	34
Kép 39 - responseban a kiválasztott játékosok hozzáadása	34
Kép 40 - teljes store/generatePack/:id végpont	35
Kép 41 - A ProtectedRoute védi az oldalakat	36
Kép 42 - isLoggedInIn useState	36

Kép 43 – checkLogin function	37
Kép 44 – átirányítás a bejelentkezéshez, vagy a komponensek renderelése	37
Kép 45 – Drag állapotok	38
Kép 46 – getPoint függvény	38
Kép 47 – startDrag függvény	39
Kép 48 – handleMove függvény	39
Kép 49 – handleUp függvény	40
Kép 50 – Event listener-ek kezelése a drag-ben	40
Kép 51 – Drag komponens visszatérési értékei	41
Kép 52 – drag useEffect dependency-k	41
Kép 53 – Gamelayout props-ok	41
Kép 54 – pozíció kiírása a slot alá	42
Kép 55 – slot-ok generálása	42
Kép 56 – játékosok hozzáadása a formációhoz	43
Kép 57 – PlayerCard props-ok	44
Kép 58 - rarityImgs	44
Kép 59 – kapus ellenőrzés és szövegek színezése	44
Kép 60 –játékos másodlagos pozícióinak összegyűjtése	45
Kép 61 – kártyadizájn renderelése	45
Kép 62 – kártya alap adatainak renderelése	46
Kép 63 – összhang csillagok renderelése	47
Kép 64 – egy stat renderelése a kártyán	47
Kép 65 – stat pozícionálásához szükséges CSS formázás	48
Kép 66 - /api/formations HTTP request-je	48
Kép 67 - /api/formations HTTP request eredménye	49
Kép 68 - /api/users/login HTTP POST request-je	49
Kép 69 - /api/users/login POST request első eredménye	50
Kép 70 - /api/users/login POST request második eredménye	50
Kép 71 - /api/users/login POST request harmadik eredménye	51
Kép 72 – betekintés a unit tesztbe	52
Kép 73 – QUnit teszteredmények	53
Kép 74 - XAMPP telepítése	56
Kép 75 - phpMyAdmin jelszó megadása	57
Kép 76 - terminál megnyitása	57
Kép 77 - split terminál	58
Kép 78 - terminálon belül mappákba belépés	58
Kép 79 - XAMPP modulok elindítása	58
Kép 80 - elindítandó service-ek	59
Kép 81 - phpMyAdmin megnyitása	59
Kép 82 - adatbázis beillesztés 1. lépés	59
Kép 83 - adatbázis beillesztés 2. lépés	60
Kép 84 - adatbázis beillesztés 3. lépés	60
Kép 85 - regisztrációs felület elérése	61
Kép 86 - regisztrációs felület	62
Kép 87 - bejelentkezési felület	62

Kép 88 - squad képernyő	63
Kép 89 - chemistry magyarázat grafika	63
Kép 90 - sbc képernyő	64
Kép 91 - feltételek menüpont	64
Kép 92 - draft formáció választó képernyő	65
Kép 93 - kapitány választás	66
Kép 94 - draft kiértékelés.....	66
Kép 95 - draft jutalmak	66
Kép 96 - my packs képernyő	67
Kép 97 - vásárolható csomagok	67
Kép 98 - csomag nyitó képernyő	68
Kép 99 - quick sell all opció	68
Kép 100 - duplicate játékosok.....	68
Kép 101 - my club képernyő	69
Kép 102 - filter funkció	69
Kép 103 - leaderboard képernyő	70
Kép 104 - milestones	71
Kép 105 - foundations	71
Kép 106 - seasonal	71
Kép 107 - settings képernyő.....	72

7. Irodalomjegyzék

- <https://youtu.be/s2skans2dP4?is=R2CN2UUH45wyEM7G> (2025.12.28)
- https://youtube.com/playlist?list=PLbKN8A2wssqUFvULAa4lJNvGkiYXDqAnE&si=9qG-bP-cMwWxwd_y (2026.01.12)
- <https://code.mu/hu/javascript/framework/react/book/prime> (2026.03.21)
- https://www.reddit.com/r/fifacardcreators/comments/1kxkbce/fc_26_rating_concepts_card_by_gamingfriedo25/mupx2r9/?context=3&share_id=iRD6mKMs8E4A2QFCSVui2&utm_content=1&utm_medium=ios_app&utm_name=ioscss&utm_source=share&utm_term=1&seeker-session=true (2025.11.03)
- https://dev.to/kazutora_hattori_66972c88/a-simple-explanation-of-the-meaning-and-usage-of-prev-prev-and-key-in-react-usestate-2h6o (2026.02.15)
- <https://medium.com/@deval93/10-react-hooks-explained-with-real-examples-2025-edition-the-guide-i-wish-i-had-3-years-ago-eobo86f761a4> (2026.04.24)
- <https://www.w3schools.com/> (2026.04.26)
- <https://developer.mozilla.org/en-US/> (2026.04.22)
- <https://react.dev/> (2026.04.26)
- <https://www.fifacm.com/> (2026.02.20)
- <https://www.futbin.com/players> (2026.03.18)
- <https://www.futwiz.com/fc26/players> (2026.03.25)
- <https://www.kaggle.com/> (2026.01.17)
- <https://expressjs.com/en/resources/middleware/multer.html> (2026.04.12)
- <https://blog.openreplay.com/multer-npm-file-upload-nodejs> (2026.04.12)

- <https://fifauteam.com/fc-25-chemistry> (2025.11.08)
- <https://fifauteam.com/fc-25-squad-rating-guide/> (2025.12.05)
- <https://chatgpt.com> (2026.04.26)
- <https://texturelabs.org> (2026.04.14)